

Sistemi Embedded

Dispensa Integrata di Studio
(Fusione e revisione dei materiali di corso)

Contents

1	Introduzione ai Sistemi Embedded	4
1.1	Cyber-Physical Systems	4
1.2	Tecnologia MOS e Circuiti Integrati	5
1.3	Un Richiamo a Unix	5
2	Circuiti Integrati: Storia, Classificazione e Fondamenti Teorici	6
2.1	Storia ed Evoluzione dei Circuiti Integrati	6
2.2	Circuiti Non Programmabili e Programmabili	6
2.3	La Macchina di Turing e i suoi Fondamenti	7
2.4	La Complessità Computazionale	8
3	Sintesi dei Circuiti Digitali	8
3.1	Il Flusso di Progettazione Top-Down	8
3.2	Sintesi Hardware vs Sintesi Software	9
3.3	Il Productivity Gap	10
3.4	I Livelli di Astrazione della Sintesi Hardware	10
3.5	Metriche di Ottimizzazione	11
3.6	Un Esempio Completo di Sintesi: l'Equazione Differenziale	11
3.6.1	Trade-off tra Area e Latenza	12
3.7	Scheduling e Binding	13
4	Sintesi FPGA e Flusso RTL	13
4.1	Livelli di Descrizione dell'Hardware	13
5	VHDL	14
5.1	Storia e Nascita del Linguaggio	14
5.2	I Livelli di VHDL	14
5.3	Entity e Architecture	15
5.4	Regole Sintattiche Generali	15
5.5	Il Tipo std_logic	15
5.6	Signal vs Variable	16
5.7	Statement Sequenziali e Concorrenziali	16
5.8	Esempi Comparati di Codice	16
5.9	Tipi Sintetizzabili	17
5.10	Logica Combinatoria e Operatori Bitwise	18
5.11	Conditional Assignment e Multiplexer	18
5.12	Segnali Interni e il Full Adder	18

5.13	Precedenza degli Operatori	19
5.14	Tri-State, Alta Impedenza e Valori Indeterminati	19
5.15	Bit Coalescing e Output Splitting	20
5.16	Sign Extension	21
5.17	I Ritardi in Simulazione	21
5.18	Costrutti di Controllo Avanzati: Case, If-Else e Generate	21
5.19	Logica Sequenziale in VHDL	23
5.19.1	Reset Sincrono e Asincrono	23
5.19.2	Registro con Enable	23
5.19.3	Latch Trasparenti	23
5.20	Organizzazione della Memoria e Decodifica degli Indirizzi	23
5.21	Divide-by-3 FSM: un Esempio Completo	24
5.22	Macchine a Stati Finiti: Moore e Mealy	24
5.23	Type Idiosyncrasies in VHDL	25
5.24	Moduli Parametrizzati (Generic)	25
5.25	Memorie in HDL	25
5.26	Testbench	26
6	FPGA nel Dettaglio	26
6.1	Logiche Programmabili e il Trade-off Economico	26
6.2	Architettura di Sistema FPGA	27
6.3	Programmabilità Fisica	27
6.4	Programmabilità Logica	28
6.5	Blocchi Logici Configurabili (CLB)	28
6.6	Architettura Xilinx Spartan-II	29
6.7	Sintesi RTL Tradizionale vs High-Level Synthesis	29
6.8	Interconnessioni e Ritardi	29
6.9	Condizionamento del Segnale e Blocchi I/O	30
7	Microprocessori e Microcontrollori	31
7.1	Componenti Generali di un Microcontrollore	31
7.2	Protocolli di Comunicazione: SPI e I2C	31
7.3	Il Watchdog Timer	32
7.4	Organizzazione della Memoria	32
7.5	Modalità di Interfaccia Periferica	33
8	Digital Signal Processors (DSP) e SoC	33
8.1	Motivazioni e Peculiarità	33
8.2	DSP su System on Chip	34
8.3	Aritmetica DSP e Unità MAC	34
8.4	Architetture di Memoria: Von Neumann e Harvard	34
8.5	Modalità di Indirizzamento DSP	35
9	Architettura MIPS	35
9.1	Il Design Quantitativo del Microprocessore	35
9.2	Evoluzione delle Architetture	36
9.3	Endianness e Allineamento	37
9.4	Modalità di Indirizzamento MIPS	37
9.5	L'Interfaccia Software/Hardware e le Istruzioni MIPS	37
9.6	L'Architettura Single-Cycle	38
9.7	L'Architettura Multi-Cycle	39

10 Gerarchia di Memoria e Cache	40
10.1 Il Principio di Località	40
10.2 Architetture di Cache	40
10.3 La Classificazione dei Miss e la Coerenza	41
10.4 Memoria Virtuale	42
11 Multiply and Accumulate (MAC)	42
11.1 Il Prodotto Complesso e i suoi Componenti	42
11.2 La Gestione della Crescita dei Bit	43
11.3 La Moltiplicazione Binaria in Hardware	43
12 Circuiti Digitali: Logica, Layout e Design Fisico	43
12.1 Dispositivi CMOS e Conduzione Complementare	43
12.2 Logica Sequenziale a Livello Transistor	44
12.3 Il Layout Fisico e le Design Rules	44
12.4 Il Physical Design del Processore	44
13 Affidabilità, Rumore e Variazioni di Processo	45
13.1 Process Corners	45
13.2 Meccanismi di Guasto e Reliability	45
14 Test dei Circuiti Integrati (Design For Testability)	46
14.1 Il Rationale del Testing	46
14.2 Fault Modeling	47
14.3 Design For Testability	47
15 Memorie e Array	48
15.1 Static RAM	48
15.2 Decoder e Memorie Grandi	48
15.3 Memorie ROM e DRAM	48
15.4 Memorie Seriali	49
16 Mappa Concettuale Riassuntiva	49

1 Introduzione ai Sistemi Embedded

L'evoluzione dei sistemi informatici ha portato progressivamente allo sviluppo di quella particolare classe di dispositivi che oggi chiamiamo *sistemi embedded*. La spinta verso questi sistemi non è nata per caso, ma è la risposta naturale a un'esigenza di mercato ben precisa: la necessità di dispositivi più piccoli, più economici, più efficienti dal punto di vista energetico e, soprattutto, progettati per svolgere un compito specifico piuttosto che una gamma generica di funzioni. Basti pensare che Linux, nella sua declinazione embedded, è diventato oggi il sistema operativo scelto per un numero impressionante di applicazioni integrate: router per la connessione a Internet, sistemi di navigazione satellitare GPS, dispositivi di archiviazione collegati in rete e moltissimi altri prodotti che usiamo quotidianamente senza nemmeno accorgerci che al loro interno gira un vero sistema operativo.

Se ci chiediamo perché si sia arrivati a questo punto, la risposta va cercata in un cambio di prospettiva più ampio: da un lato si è affermata una visione *processor-centrica*, in cui il software è diventato il vero motore dell'elaborazione delle informazioni; dall'altro la miniaturizzazione dei circuiti integrati ha reso possibile racchiudere una potenza di calcolo un tempo impensabile in spazi minuscoli. L'incontro di queste due tendenze ha dato origine ai sistemi embedded così come li conosciamo. Le ragioni che hanno guidato questo sviluppo sono essenzialmente due. La prima è la crescente **complessità funzionale**: aggiungere funzionalità avanzate a un dispositivo richiede l'integrazione di quantità sempre maggiori di software, un'operazione resa possibile dall'aumento esponenziale della densità dei semiconduttori descritto dalla celebre **Legge di Moore**, secondo cui il numero di transistor presenti su un circuito integrato raddoppia approssimativamente ogni due anni. A questa si affianca la cosiddetta *legge del ritorno accelerato*, che insieme hanno permesso lo sviluppo di dispositivi sempre più potenti e complessi. La seconda ragione è la ricerca di **efficienza e miniaturizzazione**: la combinazione ottimizzata di hardware e software tipica dei dispositivi embedded consente di ridurre drasticamente le dimensioni fisiche e i costi di produzione, un passo fondamentale per realizzare dispositivi compatti e a basso consumo energetico.

Definizione 1.1 (Sistema Embedded). *Si definisce sistema embedded (o sistema integrato) un sistema di elaborazione delle informazioni incorporato all'interno di un prodotto di dimensioni maggiori. Il problema tecnico centrale legato ai processi fisici che questi sistemi devono controllare è la gestione della concorrenza e della computazione in tempo reale.*

1.1 Cyber-Physical Systems

A partire da questa definizione si introduce un concetto più ampio, quello dei **cyber-physical systems** (CPS), ovvero sistemi informatico-fisici che rappresentano l'integrazione del calcolo con processi fisici reali. I CPS si riferiscono a sistemi ICT (Information and Communication Technologies) di nuova generazione, interconnessi attraverso l'**Internet of Things** (IoT), che permette loro di collaborare tra loro: è proprio in questo contesto che si parla comunemente di "Industria 4.0". Un CPS differisce da un tradizionale sistema di controllo digitale principalmente nella sua struttura concettuale: mentre un sistema di controllo classico si limita a ricevere un riferimento e a confrontare l'errore con il segnale di retroazione, in un CPS il "cyber", cioè la parte computazionale, è molto più avanzata e include, oltre al semplice controllo, anche funzioni di calcolo avanzate, comunicazione tra dispositivi e analisi dei dati raccolti.

A differenza dei computer generici e riprogrammabili, un sistema embedded ha compiti noti già in fase di sviluppo, che eseguirà grazie a una combinazione hardware/software studiata appositamente per quella specifica applicazione. Questo permette di ridurre l'hardware ai minimi termini, contenendo così lo spazio occupato, limitando i consumi, migliorando i tempi di elaborazione e riducendo il costo di fabbricazione. Inoltre, l'esecuzione del software è spesso vincolata al tempo reale, per permettere un controllo deterministico dei tempi di esecuzione:

in sostanza i sistemi embedded comprendono ogni tipo di calcolatore al di fuori di quelli progettati per un utilizzo di uso generico.

Se confrontiamo un sistema embedded con un processore *general purpose* emergono differenze sostanziali su più fronti:

- **Scopo e utilizzo:** un sistema embedded svolge compiti specifici all'interno di un sistema più grande, mentre un processore *general purpose* viene progettato per una vasta gamma di compiti diversi.
- **Hardware:** un sistema embedded è progettato per essere altamente efficiente dal punto di vista energetico, spesso alimentato a batteria, e dispone di risorse limitate in termini di memoria RAM e ROM e di capacità di elaborazione; un processore *general purpose*, al contrario, tende a consumare più energia ma ha accesso a risorse ben più abbondanti.
- **Architettura e design:** un sistema embedded sfrutta architetture specifiche ottimizzate per un compito, facendo uso di interfacce come GPIO, ADC, DAC, I2C, UART e SPI, mentre un *general purpose* utilizza architetture più versatili come x86 e supporta una vasta gamma di periferiche quali USB, HDMI e PCIe.
- **Software:** nei sistemi embedded si esegue software dedicato, scritto e ottimizzato per quello specifico compito.

Le applicazioni dei sistemi embedded sono molteplici e attraversano diverse aree della vita moderna. Nell'automazione di fabbrica, la tecnologia CPS/IoT è la chiave per una produzione più flessibile, favorendo il raggiungimento degli obiettivi dell'Industria 4.0. La robotica è un'area tradizionale in cui questi sistemi vengono da sempre impiegati. Nel trasporto e nella mobilità, l'elettronica in ambito automotive è ormai onnipresente, dato che le auto moderne contengono una quantità significativa di componenti elettronici. Infine, nelle smart city si adottano strategie di pianificazione urbanistica che migliorano la qualità della vita cercando di soddisfare le esigenze dei cittadini.

1.2 Tecnologia MOS e Circuiti Integrati

Alla base di tutta l'elettronica digitale moderna troviamo la tecnologia **MOS** (Metal–Oxide–Semiconductor), che consente di realizzare su un singolo chip di silicio miliardi di transistor controllabili elettricamente, rendendo possibili microprocessori, memorie e sistemi embedded complessi. Il processo produttivo parte dal silicio ultrapuro, prodotto sotto forma di lingotto monocristallino mediante tecniche come il metodo Czochralski o il Float Zone, che garantiscono una struttura cristallina quasi perfetta. Il lingotto viene poi tagliato in wafer sottili e lucidato, sui quali, attraverso processi estremamente precisi di fotolitografia, ossidazione, drogaggio ionico e deposizione di materiali, si costruiscono strati successivi di transistor e interconnessioni metalliche.

1.3 Un Richiamo a Unix

Per comprendere la genesi filosofica dei moderni sistemi operativi embedded, è utile un breve richiamo alla nascita di Unix: un sistema pensato fin dall'origine per essere portabile, modulare e componibile, filosofia che si è poi riflessa nelle derivazioni embedded come Linux embedded, capaci di adattarsi a hardware con risorse estremamente limitate mantenendo comunque un'architettura a processi, file system gerarchico e gestione della concorrenza.

2 Circuiti Integrati: Storia, Classificazione e Fondamenti Teorici

2.1 Storia ed Evoluzione dei Circuiti Integrati

A partire dagli anni '80 le aziende iniziarono a progettare sistemi elettronici integrati custom, che condensavano tutte le funzioni in un unico circuito specifico per l'applicazione: i cosiddetti **ASIC**. Questi circuiti presentavano però un grosso problema: erano costosi da realizzare e risolvevano un problema molto specifico, per cui non erano riutilizzabili in altri contesti. In generale, i microprocessori sono i dispositivi che consumano di più, mentre a parità di consumo i circuiti dedicati sono i più performanti, seppur con un costo importante. Le FPGA hanno il vantaggio di essere poco costose e di avere un rapporto consumo/performance molto buono.

Nell'analisi dei costi di un circuito integrato è utile distinguere due categorie: i **costi fissi** (*Fixed Costs*), che non dipendono dal numero di chip prodotti e comprendono la formazione del personale, gli strumenti hardware e software, i costi di progettazione, il design dei test e il modello di profitto; e i **costi variabili** (*Variable Costs*), legati alle materie prime come i wafer di silicio, i materiali di consumo, i costi di produzione, il packaging e il testing. Se indichiamo con N il numero di dispositivi prodotti, il costo totale si esprime come

$$\text{Costo Totale} = FC + VC \cdot N :$$

se produciamo pochi pezzi, a dominare è il costo fisso; se ne produciamo tantissimi, a dominare è il costo variabile. Questo semplice ragionamento economico spiega perché, con l'aumentare della complessità dei circuiti e quindi dei costi fissi di progettazione (maschere, verifica, test), la finestra temporale ed economica in cui conviene realizzare circuiti integrati completamente custom si sia progressivamente ristretta, spingendo verso soluzioni alternative come i circuiti semi-custom (MGA e CBIC) e i circuiti programmabili (PLD e FPGA). Un fattore ulteriore da considerare è il *time-to-market*: un ritardo nell'uscita di un prodotto riduce il profitto totale, perché il mercato ha una finestra temporale limitata prima della fine del ciclo di vita del prodotto stesso, e le vendite perse nella fase iniziale non vengono più recuperate.

2.2 Circuiti Non Programmabili e Programmabili

I circuiti integrati si dividono in due grandi categorie. I **circuiti non programmabili** hanno una funzionalità definita durante la fase di progettazione e produzione, e una volta fabbricati il loro comportamento non può più essere modificato. Rientrano in questa categoria:

- i **circuiti custom**, meglio noti come **ASIC**, progettati su misura per una specifica applicazione ma con costi di progettazione e realizzazione molto elevati;
- i **circuiti semi-custom**, che offrono un compromesso tra circuiti standard e custom. Fra questi troviamo gli **MGA** (Masked Gate Arrays), a loro volta suddivisi in *Channeled Gate Array* (struttura a canali predefiniti per le interconnessioni, dove il concetto di standard-cell è esteso a tutto il circuito personalizzando solo lo spazio predefinito tra le righe di celle base) e *Channelless Gate Array* (o *Sea of Gates*: non ci sono canali predefiniti, l'intera superficie è coperta da un'alta densità di transistor e le interconnessioni vengono inserite sopra tramite strati metallici). Esiste anche lo **Structured Gate Array**, un compromesso tra Gate Array tradizionali e circuiti completamente custom, che comprende blocchi di funzionalità predefinite (memorie, processori, blocchi analogici) integrati con blocchi programmabili di transistor.

Un'altra realizzazione semi-custom molto diffusa sono le **Standard Cells** o **CBIC** (Cell-Based IC): in questo caso il chip non viene progettato transistor per transistor, ma tramite una libreria di celle standard pre-progettate e ottimizzate che implementano funzioni tipiche come

porte logiche AND, OR, NOT, latch, flip-flop e multiplexer. Queste celle sono già caratterizzate dal costruttore in termini di ritardi, consumo e area, e vengono posizionate e collegate automaticamente per realizzare la funzione desiderata. Il costo fisso delle standard-cells si abbatta molto rispetto ai circuiti custom, mantenendo comunque un'elevata personalizzazione tramite la realizzazione delle interconnessioni tra i blocchi fissi. In questo contesto si parla anche di **Fixed Block**: parti di un circuito integrato che rappresentano funzionalità predefinite e complesse (memoria RAM/ROM, processori, blocchi analogici), con dimensioni e posizioni predeterminate nel layout, a differenza delle standard cells che possono essere ridimensionate e riposizionate liberamente.

I **circuiti programmabili**, invece, hanno una funzionalità che può essere definita o modificata dall'utente dopo la produzione, il che li rende ideali per una vasta gamma di applicazioni. Comprendono i **PLD** (Programmable Logic Devices), dispositivi programmabili per eseguire funzioni logiche semplici, e le **FPGA** (Field-Programmable Gate Arrays), circuiti integrati programmabili avanzati capaci di implementare funzioni logiche complesse. Per connessione programmabile si intende, sia nei PLD che negli FPGA, una rete configurabile di interruttori elettronici che stabilisce percorsi logici tra i blocchi del dispositivo: il segnale digitale 0 o 1 è rappresentato da livelli di tensione, e queste connessioni possono essere programmate e riprogrammate per modificare il comportamento del circuito. È bene sottolineare che, in questo ambito, programmabilità *non* significa esecuzione di software: una FPGA non esegue un programma come farebbe una CPU, ma viene riconfigurata fisicamente per diventare un circuito dedicato.

2.3 La Macchina di Turing e i suoi Fondamenti

All'inizio del 1900, i matematici guidati da Hilbert cercarono di formalizzare l'intera matematica in un sistema assiomatico rigoroso, nel quale tutte le branche derivassero da un insieme di assiomi fondamentali. Kurt Gödel, con il suo **Teorema di Incompletezza**, dimostrò però due fatti sorprendenti: in ogni sistema matematico assiomatico sufficientemente potente da contenere l'aritmetica elementare esistono proposizioni indecidibili, cioè che non possono essere né dimostrate né confutate all'interno del sistema stesso pur essendo vere o false indipendentemente; e la consistenza di un sistema matematico F non può essere dimostrata all'interno dello stesso sistema F , il che implica che non è possibile garantire la totale affidabilità del sistema basandosi esclusivamente sui suoi assiomi e regole interne. Questo risultato ha avuto un profondo impatto sulla logica e sulla filosofia, dimostrando che la matematica non può essere completamente ridotta a un insieme di regole meccaniche.

Fu Alan Turing ad affrontare il problema della formalizzazione della computabilità, elaborando la **Macchina di Turing**, un modello teorico capace di rappresentare qualsiasi processo algoritmico. Concetti chiave associati a questo modello sono la *congettura di Church-Turing*, secondo cui qualunque problema computazionale che ammette una soluzione algoritmica può essere risolto da una macchina automatica, e il concetto di *effettiva calcolabilità*, per cui una funzione si dice effettivamente calcolabile se i suoi valori possono essere determinati attraverso un processo puramente meccanico.

Definizione 2.1 (Macchina di Turing). *La macchina di Turing è formata da tre componenti: il **nastro** (la memoria), una sequenza di celle considerata infinita, ciascuna delle quali può contenere un simbolo appartenente a un alfabeto finito; la **testina** di lettura/scrittura, che legge il simbolo corrente, può sovrascriverlo o spostarsi di una cella a destra o sinistra; e la **control unit** (la FSM), definita da una quintupla di elementi che comprende lo stato attuale, il simbolo letto, lo stato successivo, il simbolo da scrivere e la direzione di movimento della testina.*

La macchina opera su intervalli discreti di tempo: a ogni istante il suo stato attuale e le

sue azioni future dipendono dallo stato precedente e dal simbolo letto. Un microprocessore può essere considerato una macchina di Turing grazie alla sua dimensione: sebbene abbia memoria finita, essa è sufficientemente grande da poter essere assimilata al nastro infinito; la testina è assimilabile all'accesso alla RAM in lettura/scrittura, mentre l'unità di elaborazione corrisponde all'ISA (Instruction Set Architecture). Di conseguenza, con un microprocessore e il programma opportuno possiamo risolvere qualunque problema computazionale che ammetta una soluzione algoritmica. Quando un modello computazionale ha capacità di soluzione dei problemi pari a quelle di una macchina di Turing, si dice che è **Turing-completo**: un esempio significativo è un modello computazionale basato su porte logiche NAND, che essendo funzionalmente completo può rappresentare qualsiasi funzione logica booleana, e combinando più porte NAND si costruiscono componenti complessi (ALU, registri, multiplexer) che formano un processore Turing-completo.

2.4 La Complessità Computazionale

Un altro concetto teorico fondamentale è quello di **complessità computazionale**, che si occupa di analizzare come crescono il tempo e lo spazio necessari per risolvere un problema mediante un algoritmo all'aumentare della dimensione dell'input N . Questa crescita si esprime tramite la notazione asintotica Big-O: $O(\log N)$ per la crescita logaritmica, $O(N)$ per quella lineare, $O(N^2)$ per quella polinomiale e $O(2^N)$ per quella esponenziale. In base a questa classificazione si distinguono tre classi di problemi. I problemi **polinomiali** (P) sono algoritmi efficienti, risolvibili in tempi ragionevoli con risorse limitate, la cui complessità cresce secondo una funzione polinomiale (l'esempio classico è il Merge Sort). I problemi **nondeterministici polinomiali** (NP) diventano rapidamente impraticabili, con un calcolo che cresce in modo esponenziale. Infine i problemi **NP-completi** sono i più difficili della classe NP : se si trovasse che un problema NP-completo è risolvibile in tempo polinomiale, tutti i problemi della classe NP diverrebbero improvvisamente polinomiali.

3 Sintesi dei Circuiti Digitali

3.1 Il Flusso di Progettazione Top-Down

Esistono due filosofie contrapposte nella progettazione: il **top-down**, in cui si parte dal problema e si scende progressivamente fino a raggiungere il dettaglio più semplice per realizzare la soluzione, e il **bottom-up**, in cui si parte dal programmare la funzione elementare e la si assembla man mano fino a costruire il sistema completo. Nel contesto degli embedded systems si adotta tipicamente un approccio top-down: la progettazione procede dal livello più astratto a quello più concreto, partendo dall'*idea*, cioè dalla definizione di cosa il sistema deve fare, per poi passare alla *formalization*, in cui l'idea viene tradotta in requisiti funzionali, temporali e di consumo. Si prosegue con la *block structure*, dove il sistema viene suddiviso in blocchi hardware e software (microcontrollore, sensori, attuatori, moduli software, eventuale RTOS), e con il *detailed design*, in cui si progettano nel dettaglio i singoli blocchi come schemi elettrici, driver, task, interrupt, macchine a stati e gestione del timing. Si arriva quindi alla fase di *synthesis*, la traduzione del progetto in una forma realizzabile tramite compilazione del codice, sintesi logica e mapping hardware/software, e infine alla *realization*, cioè l'implementazione finale del sistema su hardware reale. Durante tutte le fasi è fondamentale la *verification*, effettuata a ogni livello per controllare che le scelte progettuali rispettino le specifiche definite nei livelli superiori, individuando gli errori il prima possibile.

Nei linguaggi di programmazione esiste una corrispondenza biunivoca tra costruito sintattico e la sua semantica: cioè la semantica espressa da un determinato costruito non è

interpretabile, ma ha un significato ben definito. Il linguaggio hardware, in questo senso, è la creazione di un algoritmo volto alla descrizione dell'hardware stesso.

3.2 Sintesi Hardware vs Sintesi Software

Nello schema di progetto di un sistema tipicamente embedded o hardware-oriented, il processo si suddivide in fasi consecutive: si parte dall'idea e si procede con il *modeling* del sistema, seguito dalla *synthesis & optimization*, in cui il modello viene trasformato in una soluzione implementabile ed efficiente, e dalla *validation*, che verifica la correttezza funzionale rispetto alle specifiche. Successivamente si passa al *testing*, dove il sistema viene sottoposto a test strutturati; se il progetto riguarda componenti hardware, si entra poi nella fase di *fabrication* (produzione delle maschere e dei wafer) e infine nella fase di *packaging*, che include il taglio e l'incapsulamento finale del componente.

Nella sintesi software l'obiettivo è mappare una descrizione comportamentale astratta, come il codice sorgente C o C++, su una risorsa hardware fissa e generale, il processore. Poiché le risorse di calcolo sono limitate e condivise, il processo impone un binding temporale: le operazioni vengono serializzate nel tempo per essere eseguite sequenzialmente dall'automa. È interessante osservare che il processo di automatizzazione dell'implementazione esiste, ma si perde progressivamente controllo sul risultato finale (il componente scritto nel silicio) a seconda del livello di astrazione raggiunto: più specifiche di basso livello si implementano, più il componente sul silicio sarà simile a quello pensato in origine, mentre esistono livelli di astrazione che è sconsigliabile implementare per un uso generico (ad esempio specificare solo la necessità di avere un adder, senza precisare quale tipo).

L'interfaccia critica che funge da contratto tra hardware e software è l'**ISA** (Instruction Set Architecture), che definisce le operazioni primitive che la macchina può eseguire. Il processo di compilazione avviene in tre fasi distinte:

- **Front-end** (analisi e astrazione): si esegue l'analisi lessicale, sintattica e semantica per verificare la correttezza grammaticale del codice, generando una prima rappresentazione intermedia, solitamente un *Abstract Syntax Tree* (AST), che descrive la struttura logica del programma.
- **Middle-end** (ottimizzazione indipendente dall'architettura): si trasforma l'AST in una *Intermediate Representation* (IR) o in un *Control Data Flow Graph* (CDFG), eseguendo ottimizzazioni matematiche e logiche, come la rimozione del codice morto o la semplificazione dei cicli, senza preoccuparsi ancora di quale processore verrà usato.
- **Back-end** (generazione del codice e mapping): si traduce l'IR nell'Instruction Set specifico del target, si esegue la *Register Allocation*, cioè si mappano le variabili infinite del programma sul numero finito di registri fisici della CPU, e si effettua l'*Instruction Scheduling*, riordinando le istruzioni per massimizzare l'efficienza della pipeline.

Il **Datapath** è, in quest'ottica, il cammino che i dati e le istruzioni devono seguire per essere lavorati. Anche nella sintesi hardware si specifica un comportamento, ma qui l'hardware non è dato, bensì va costruito specificando cosa si vuole realizzare: la differenza rispetto alla sintesi software risiede dunque nel livello di astrazione più basso. La sintesi hardware comporta, oltre al ricorso a dispositivi progettati ad hoc, anche la scelta di dispositivi eventualmente già esistenti (semi-custom) e la scelta dei particolari microprocessori da utilizzare; la selezione di questi dispositivi influenza a sua volta la generazione del software. Collegando il concetto di datapath al diagramma front-end/intermediate form/back-end, il processo può essere interpretato in modo chiaro anche per la sintesi hardware: nel front-end si analizza la specifica comportamentale del sistema, senza ancora stabilire come verrà realizzato; nella

intermediate form il comportamento viene riorganizzato e ottimizzato, individuando le operazioni fondamentali e i flussi di dati, ed è in questa fase che inizia a emergere la struttura del datapath; nel back-end, infine, avviene la vera e propria costruzione del datapath, con la scelta dei componenti fisici e il loro collegamento.

Occorre anche definire il concetto di **piattaforma**: a differenza del datapath, che descrive il cammino fisico del dato tra componenti, la piattaforma è l'astrazione hardware-software creata per interfacciarsi con i livelli applicativi superiori, fungendo da infrastruttura di comunicazione e gestione delle risorse.

Nel processo di progetto ci sono diverse fasi con specifiche funzioni da assolvere: deve essere possibile codificare il modello (avere un entry point in cui specificare la funzionalità), validarlo e debuggarlo attraverso la simulazione al livello della codifica, scomporre l'idea in blocchi funzionali tramite la scelta delle opzioni architetturali, e infine giungere al progetto esecutivo, dove si parla di **sintesi** (che genera l'hardware che realizza quel funzionamento) o di **co-sintesi** (data dal fatto che si può decidere di realizzare porzioni in hardware e porzioni in software su un microprocessore dedicato).

3.3 Il Productivity Gap

Il motivo per cui si è passati progressivamente da un flusso di progetto puramente hardware a uno più orientato al software è descritto dal cosiddetto **productivity gap**: il numero di transistor resi disponibili dalla tecnologia cresce molto più rapidamente del numero di transistor che un linguaggio come VHDL permette di gestire, dato il suo livello di astrazione. Mentre la tecnologia consente di integrare sempre più logica sul silicio, la progettazione manuale a basso livello non scala allo stesso ritmo, creando un divario tra la complessità hardware disponibile e quella realmente gestibile dal progettista. Per colmare questo divario sono nati i software **EDA** (Electronic Design Automation), che consentono di descrivere il sistema a un livello di astrazione più alto tramite un modello HDL, delegando agli strumenti automatici la traduzione verso livelli più bassi, occupandosi della sintesi logica, della generazione dell'hardware fisico e dell'ottimizzazione del progetto rispetto a metriche specifiche come area, consumo di potenza, prestazioni o costo.

3.4 I Livelli di Astrazione della Sintesi Hardware

La sintesi hardware si caratterizza per due assi indipendenti, uno dei quali è proprio l'asse dell'astrazione. Si distinguono tre livelli di astrazione, dal basso verso l'alto. Il **livello geometrico** rappresenta una leggera astrazione del livello fisico. Il **livello logico**, descritto attraverso porte logiche, è quello a cui si minimizza l'area in presenza di vincoli sul ritardo di propagazione, oppure si minimizza il ritardo di propagazione in presenza di vincoli sull'area. Il **livello architetturale**, infine, è quello a cui si determina il cycle-time, si minimizza l'area in presenza di vincoli sulla latenza, e si minimizza la latenza in presenza di vincoli sull'area.

Ciascuno di questi livelli può essere visto secondo due prospettive: la **vista strutturale**, che al livello logico corrisponde alla mappatura di porte logiche e al livello architetturale a uno schema a blocchi; e la **vista comportamentale**, che al livello logico corrisponde a una macchina a stati finiti e al livello architetturale al codice sorgente. Un sistema embedded può essere descritto secondo diverse *view*, che permettono di analizzare e progettare lo stesso sistema a diversi livelli di dettaglio, mantenendo separati comportamento e struttura. La vista comportamentale descrive cosa fa il sistema, senza specificare come è realizzato fisicamente, concentrandosi su sequenza delle operazioni, algoritmi e flusso di controllo. La vista strutturale descrive invece come è fatto il sistema, cioè i componenti che lo compongono e le loro interconnessioni, includendo blocchi funzionali come ALU, unità di controllo, memoria e bus.

A ogni livello di astrazione corrisponde un livello di progetto, e ogni passaggio da un livello

superiore a uno inferiore corrisponde a una fase di sintesi, intesa come fase di ottimizzazione del progetto finalizzata a soddisfare specifici vincoli. La **sintesi architetturale** parte da una descrizione del comportamento architetturale e determina la struttura macroscopica del sistema, definendo i macro-blocchi principali e le loro interconnessioni. La **sintesi logica** parte da una descrizione del comportamento logico e produce la struttura microscopica del sistema, espressa in termini di porte logiche. La **sintesi geometrica**, infine, riguarda la realizzazione fisica del circuito e determina il layout, cioè la definizione geometrica delle porte logiche, la loro posizione sul chip e le interconnessioni fisiche. Durante queste fasi si passa progressivamente dalla behavioral view alla structural e alla physical view, mantenendo invariato il comportamento del sistema ma raffinando sempre di più la descrizione.

3.5 Metriche di Ottimizzazione

Le principali metriche considerate nella sintesi dei circuiti integrati sono l'**area** occupata, una proprietà estensiva (se un circuito svolge il doppio delle funzioni, occupa approssimativamente il doppio dell'area); la **performance**, che indica quanto velocemente il sistema può operare e non ha una definizione univoca, poiché dipende dal tipo di circuito: nei circuiti combinatori è descritta dal ritardo di propagazione e dal cycle-time, nei circuiti sequenziali dalla **latenza** (il tempo che intercorre tra il momento in cui i dati sono validi e quello in cui le uscite riflettono il cambiamento di stato), e nei circuiti pipelined dal **throughput**, ovvero il numero di risultati prodotti per unità di tempo (nel caso delle pipeline, la latenza rimane costante mentre aumenta il throughput, per cui la metrica rilevante diventa proprio quest'ultima). Vi sono poi la **testabilità**, che misura quanto facilmente il circuito può essere testato per individuare eventuali guasti, e la **potenza dissipata**, che indica l'energia consumata durante il funzionamento.

Per formalizzare il processo di ottimizzazione, si introducono lo **spazio di progettazione** S , che include tutte le possibili implementazioni che soddisfano il comportamento desiderato, e lo **spazio delle funzioni di valutazione** E , ottenuto applicando funzioni di valutazione (le metriche di progetto) a ogni punto di S ; le **metriche di progetto** N sono i criteri utilizzati per valutare le implementazioni, e il loro numero determina la dimensione dello spazio E . Dato lo spazio S e il corrispondente spazio E , la scelta dell'implementazione non è univoca, e serve un processo di ricerca dell'implementazione ottima tra quelle funzionalmente corrette: un'implementazione ottima corrisponde a un minimo di una funzione di costo definita sulle metriche di progetto. Dal punto di vista della complessità algoritmica, però, questo processo di ottimizzazione è intrattabile: il problema è multidimensionale e coinvolge un numero elevato di variabili, rendendo impossibile una soluzione ottima tramite algoritmi standard. Per questo motivo si ricercano soluzioni sub-ottimali, scomponendo il problema in sotto-problemi di dimensione inferiore e adottando approcci euristici.

3.6 Un Esempio Completo di Sintesi: l'Equazione Differenziale

Per rendere concreto tutto il ragionamento fin qui svolto, consideriamo un esempio classico: come un'equazione differenziale del secondo ordine possa essere trasformata in un algoritmo iterativo implementabile come hardware, o come software embedded. L'equazione da risolvere è

$$y'' + 3xy' + 3y = 0$$

con condizioni iniziali $x(0) = 0$, $y(0) = y_0$, $y'(0) = u_0$, per $x \in [0, a]$. Per semplificare il problema si introduce la variabile ausiliaria $u = y'$: in questo modo l'equazione del secondo ordine viene trasformata in un sistema di equazioni del primo ordine, $\frac{dy}{dx} = u$ e $\frac{du}{dx} + 3xu + 3y = 0$. Passando dal continuo al discreto tramite un passo di integrazione dx , le derivate vengono approssimate tramite incrementi: $u \approx u_0 - (3xu + 3y)dx$ e $y \approx y_0 + u dx$. Queste equazioni

discrete vengono poi usate in modo iterativo: $x' = x + dx$, $u' = u - (3xu dx) - (3y dx)$, $y' = y + u dx$, dopo di che i nuovi valori diventano quelli correnti ($x \leftarrow x'$, $u \leftarrow u'$, $y \leftarrow y'$).

Questo esempio mostra come un problema matematico continuo possa essere riscritto come sequenza di operazioni discrete, descritta come comportamento (HDL) e poi sintetizzata in hardware sotto forma di FSM più datapath. Osservando le operazioni richieste, ci accorgiamo che servono almeno un moltiplicatore, una ALU (dato che servono sia sommatore che sottrattori) e un'unità di controllo/memoria per salvare i risultati parziali. Per capire quante risorse impiegare in parallelo, si passa da una rappresentazione testuale sequenziale a una rappresentazione funzionale: un grafo di esecuzione, o **Data Flow Graph**, che è un grafo aciclico diretto (DAG) con un punto di partenza rappresentato da un'istruzione NOP. Da questa rappresentazione emerge il parallelismo intrinseco del problema, e quindi anche il numero di componenti necessari.

3.6.1 Trade-off tra Area e Latenza

Per scegliere il numero “ottimale” di componenti bisogna valutare il costo della soluzione sia in area sia in latenza: più risorse si aggiungono, più si possono eseguire operazioni in parallelo con una potenziale riduzione della latenza, ma allo stesso tempo aumenta l'area occupata. La latenza è l'intervallo di tempo che passa da quando gli ingressi di un blocco sono validi a quando lo sono le uscite corrispondenti; per calcolarla occorre capire l'ordine delle operazioni e quante di esse possono essere eseguite in parallelo a ogni step temporale.

Assumendo come costi semplificati un'area di 5 e una latenza di 1 per il moltiplicatore, un'area di 1 e latenza 1 per la ALU, e un'area di 1 e latenza 0 per l'unità di controllo/memoria (l'area è una proprietà estensiva: raddoppiando il numero di componenti raddoppia il costo), possiamo confrontare quattro configurazioni:

- **Soluzione (1,1)** – 1 moltiplicatore, 1 ALU: $Area = 5+1+1 = 7$. Con un solo moltiplicatore e una sola ALU, a ogni colpo di clock si esegue al più una moltiplicazione e una operazione ALU; lo scheduling delle operazioni porta a una latenza pari a 7.
- **Soluzione (2,1)** – 2 moltiplicatori, 1 ALU: $Area = 10 + 1 + 1 = 12$. Potendo eseguire due moltiplicazioni in parallelo a ogni colpo di clock, pur restando serializzate le operazioni ALU su una sola unità, la latenza si riduce a 5.
- **Soluzione (1,2)** – 1 moltiplicatore, 2 ALU: $Area = 5+2+1 = 8$. Nonostante il parallelismo sulle ALU, la latenza resta vincolata dal numero di moltiplicazioni non accelerabili con un solo moltiplicatore, risultando pari a 7.
- **Soluzione (2,2)** – 2 moltiplicatori, 2 ALU: $Area = 10+2+1 = 13$. Rappresenta il massimo parallelismo possibile tra quelle considerate: la latenza scende al minimo, pari a 4.

Rappresentando le diverse soluzioni in un grafico area-latenza, è possibile confrontarle in modo oggettivo. Dal confronto emergono soluzioni che sono oggettivamente peggiori di altre, cioè soluzioni per cui esiste almeno un'altra configurazione con area minore e latenza minore: questi punti vengono detti **dominati** e possono essere eliminati dal processo di scelta. Nel nostro esempio, la soluzione (1,2) è un punto non Pareto perché ha la stessa latenza di (1,1) ma un'area maggiore. Una volta eliminati i punti dominati, si ottiene la **curva di Pareto** (o frontiera di Pareto), l'insieme delle soluzioni per cui non è possibile migliorare una metrica senza peggiorarne un'altra, e che rappresenta l'insieme delle soluzioni ammissibili nel processo di ottimizzazione. La scelta finale tra i punti di Pareto dipende quindi dai vincoli di progetto: se il vincolo principale è la latenza si sceglierà un punto, se invece è l'area se ne sceglierà un altro.

3.7 Scheduling e Binding

Il **Non Scheduled Execution Graph** rappresenta l'insieme delle operazioni da eseguire e delle loro dipendenze logiche, senza indicare a quale istante di clock esse verranno eseguite: mostra solo l'ordine parziale imposto dalle dipendenze dei dati, non uno scheduling temporale. Con il termine **scheduling** si intende proprio il processo di determinare a quale istante di clock deve essere eseguita una determinata operazione, assegnando quindi a ogni nodo del grafo un tempo di esecuzione rispettando le dipendenze. Lo scheduling può essere effettuato senza vincoli sulle risorse (risorse infinite) oppure con vincoli sulle risorse (numero limitato di moltiplicatori, ALU, ecc.).

Esistono tre tipi di scheduling. Lo **ASAP** (*As Soon As Possible*) esegue le operazioni il prima possibile, schedulando i vertici a partire dal primo, assegnando a ciascuno il tempo di esecuzione come il massimo tra quelli già schedulati sommato al proprio ritardo di propagazione, senza tener conto di vincoli sul numero di risorse disponibili. Lo **ALAP** (*As Late As Possible*) assegna invece a ogni operazione l'istante di clock più tardivo possibile senza violare le dipendenze del grafo e fissata una latenza finale, procedendo all'indietro nel tempo a partire dall'ultimo vertice del grafo. Il **resource-constrained scheduling**, infine, viene eseguito dopo aver determinato uno scheduling ASAP o ALAP, che forniscono i limiti temporali entro cui le operazioni possono essere collocate: assumendo un numero limitato di risorse hardware, il grafo viene riallocato nel tempo per rispettare i vincoli disponibili, posticipando alcune operazioni rispetto allo scheduling ASAP per evitare la sovrapposizione nell'uso delle risorse. Questo tipo di scheduling produce uno scheduling realizzabile in hardware ed è il passaggio che lo rende compatibile con la successiva fase di resource binding.

Il **binding** è la fase del progetto in cui si decide come associare gli elementi del comportamento (le operazioni) agli elementi strutturali (le risorse hardware): dopo lo scheduling, il binding stabilisce chi fa cosa nel circuito. Con il *resource binding* si assegnano le operazioni del grafo schedulato alle risorse hardware disponibili, e ogni risorsa esegue nel tempo più operazioni diverse, secondo quanto stabilito dallo scheduling. L'assegnazione delle operazioni alle risorse è gestita tramite una macchina a stati: ogni stato della FSM corrisponde a uno o più istanti di clock e specifica quale operazione deve essere eseguita, su quale risorsa, e con quali ingressi e uscite.

4 Sintesi FPGA e Flusso RTL

4.1 Livelli di Descrizione dell'Hardware

La sintesi hardware trasforma una specifica comportamentale nell'hardware che la implementa: la specifica in ingresso deve indicare cosa il circuito deve fare, ma non come deve essere realizzato fisicamente. Si distingue tra **Abstract Behavior**, che descrive il comportamento del circuito in termini di variabili lette e scritte, condizioni di lettura e scrittura, valori temporanei, valori finali delle uscite e relazioni temporali, senza contenere informazioni sulla struttura del circuito; e il **Control-Flow Behavior**, che descrive il comportamento in termini di registri, logica combinatoria, reazioni del sistema e ordine di esecuzione delle operazioni. Il **Datapath**, in questo contesto, è la catena di risorse hardware (ALU, moltiplicatori, registri, ecc.) necessarie per eseguire le operazioni richieste dal comportamento.

Vi sono diversi livelli di astrazione nel descrivere un circuito: il **gate level**, descrizione a livello di porte logiche; il **logic level**, simile al gate level ma in termini di funzioni booleane; e il **register-transfer level** (RTL), che descrive i trasferimenti di dati tra registri e le operazioni combinatorie. Quest'ultimo include una parte comportamentale, il *register-transfer behavior*, che descrive il comportamento del circuito a livello RTL rappresentando solo le transazioni visibili a questo livello (con i segnali di controllo dati per impliciti o espressi a un livello astratto),

e una parte strutturale, la *register-transfer structure*, che descrive il circuito attraverso una descrizione strutturale con registri, operatori funzionali e loro interconnessioni esplicitamente specificati.

Idealmente, la sintesi mira a massimizzare la velocità, minimizzare l'area e l'occupazione di risorse, minimizzare i consumi di potenza, ridurre il tempo di progettazione, e massimizzare affidabilità e testabilità del circuito. Il processo è però soggetto a diversi vincoli: limiti tecnologici (ad esempio l'assenza di tristati o memoria integrata), ritardi temporali tra eventi, limiti dell'area, numero di pin disponibili, limiti sul tempo di esecuzione, e vincoli di affidabilità e testabilità.

La sintesi si compone di più passi: la sintesi propriamente detta dai livelli di astrazione superiori a quelli più bassi (che può avvenire manualmente o automaticamente), l'allocazione delle risorse con relative ottimizzazioni, la *design transformation* per soddisfare i vincoli, la composizione o decomposizione dei blocchi funzionali per corrispondere ai blocchi tecnologici disponibili, lo scheduling per assegnare gli istanti di tempo alle operazioni, e infine il binding, cioè l'assegnazione delle operazioni alle risorse disponibili.

Si distinguono tre tipi di sintesi in cascata. La **sintesi comportamentale** traduce il comportamento astratto e algoritmico in una rappresentazione a flusso di dati. La **sintesi RTL** converte questa rappresentazione in una a livello di trasferimento tra registri. La **sintesi logica**, infine, converte la rappresentazione RTL in una logica basata su porte. Il flusso completo attraversa quindi quattro passaggi: dal comportamentale al comportamento schedulato (assegnando a ogni operazione un istante di clock tramite ASAP, ALAP o vincolato dalle risorse), dal comportamento schedulato al datapath behavior (introducendo registri, definendo gli operatori attivi a ciascun clock ed esplicitando il flusso dei dati), dal datapath behavior alla RTL (dove registri e blocchi funzionali sono esplicitamente definiti e il controllo è espresso tramite segnali che abilitano selezioni e caricamenti), e infine dalla RTL alla struttura logica (dove i registri diventano flip-flop e gli operatori diventano reti di porte logiche).

5 VHDL

5.1 Storia e Nascita del Linguaggio

VHDL, acronimo di “Very High Speed Integrated Circuits HDL”, è un linguaggio di descrizione hardware concepito attorno al 1980 per rispondere a specifiche esigenze del settore tecnologico. Nasce con l'obiettivo di standardizzare i metodi di progettazione e unificare i vari dialetti HDL esistenti in un unico linguaggio, migliorando la portabilità dei progetti tra diversi strumenti EDA. Grazie a VHDL, il tempo di progettazione dei circuiti digitali è stato ridotto notevolmente: un processo che richiedeva da 6 a 18 mesi è stato compresso attraverso un approccio più efficiente. La necessità degli HDL è emersa proprio a causa del productivity gap descritto in precedenza: per superare questa limitazione, il settore ha adottato una nuova prospettiva, passando dalla progettazione a livello di porte logiche a livelli di astrazione più elevati.

Nel giugno del 1981, durante un workshop tenutosi a Woods Hole, Massachusetts, esponenti del governo statunitense e della comunità accademica definirono le caratteristiche dei Very High Speed Integrated Circuits. Nel luglio del 1983, DARPA, in collaborazione con Intermetrics, IBM e Texas Instruments, firmò un contratto per lo sviluppo di VHDL. Nell'agosto del 1985 venne rilasciata la versione 7.2, e nel dicembre 1987 il linguaggio fu ufficialmente riconosciuto come standard IEEE; sono seguiti aggiornamenti rilevanti come le versioni del 1993 e del 2008.

5.2 I Livelli di VHDL

La struttura di VHDL si articola in tre livelli principali. Il **VHDL for Specification** è dedicato alla descrizione generale del design per verificare il comportamento funzionale del circuito

hardware, concentrandosi sugli aspetti logici e comportamentali. Il **VHDL for Simulation** è scritto con l'obiettivo di consentire una simulazione accurata del circuito, permettendo di testare e verificare come esso risponderà in diverse condizioni prima della realizzazione fisica. Il **VHDL for Synthesis**, infine, è pensato per la generazione del circuito fisico: il codice è ottimizzato per essere interpretato e convertito in hardware reale, tipicamente FPGA o ASIC, e include solo istruzioni traducibili in componenti hardware effettivi.

5.3 Entity e Architecture

In VHDL, la struttura di un design è composta da due componenti fondamentali. L'**Entity** rappresenta l'interfaccia del blocco hardware, stabilendo le connessioni con l'esterno: qui vengono definite le porte (input e output) e i tipi di segnale che il circuito può ricevere o inviare, specificando cosa il circuito può fare senza descrivere come lo realizza. L'**Architecture**, invece, contiene la descrizione funzionale e strutturale di come il circuito implementa il comportamento definito nell'entity: qui vengono specificate le operazioni logiche, i processi e le relazioni tra i segnali interni. L'architecture può essere descritta a diversi livelli di astrazione, come logico o gate-level, oppure a un livello più alto come l'RTL.

Accanto a VHDL, l'altro grande linguaggio di descrizione hardware è **Verilog/SystemVerilog**. Verilog nasce nel 1984 come linguaggio di simulazione dei circuiti logici, diventa standard IEEE nel 1995, e nel 2005 viene esteso in SystemVerilog (IEEE 1800), introducendo costrutti più moderni e funzionalità avanzate per la verifica. Oggi SystemVerilog è lo standard dominante nell'industria commerciale, mentre VHDL rimane molto diffuso in ambito europeo, militare e universitario. Dal punto di vista concettuale, entrambi i linguaggi descrivono lo stesso tipo di oggetti fisici: blocchi hardware con ingressi e uscite, chiamati **module** in SystemVerilog e **entity** in VHDL. In entrambi i casi è possibile descrivere un circuito in modo comportamentale, specificando cosa deve fare, oppure strutturale, specificando come è costruito a partire da blocchi più semplici. La differenza principale tra i due linguaggi non è quindi nel tipo di hardware descrivibile, ma nella filosofia: VHDL è più rigoroso, fortemente tipizzato e vicino a una descrizione formale, mentre SystemVerilog è più compatto, flessibile e orientato alla produttività e alla verifica.

5.4 Regole Sintattiche Generali

VHDL è un linguaggio case-insensitive: **databus**, **DataBus** e **DATABUS** fanno riferimento allo stesso identificatore. Per essere validi, nomi ed etichette devono iniziare con una lettera, possono contenere lettere, cifre e underscore singoli (non sono ammessi due underscore consecutivi), non possono contenere simboli di punteggiatura e devono essere univoci all'interno della stessa entity o architecture. Non ci sono regole convenzionali obbligatorie per la formattazione, ma è buona pratica essere ordinati e mantenere un file separato per ogni entity. I commenti iniziano con `--` e si estendono fino a fine riga; non esistono commenti a blocco.

5.5 Il Tipo `std_logic`

Il tipo **BIT** è limitato a due valori logici, '0' e '1'. Tuttavia, nella progettazione digitale ci sono situazioni in cui è necessario rappresentare condizioni più complesse: per questo si raccomanda di utilizzare **std_logic** per le porte delle entità, un tipo che permette di rappresentare non solo '0' e '1' (i cosiddetti segnali forti, forniti da un componente attivo) ma anche una varietà di stati aggiuntivi. Il tipo **std_ulogic** è simile ma rappresenta un singolo bit con una restrizione in più: può assumere solo uno stato alla volta, ed è generalmente utilizzato in contesti dove serve un controllo più rigoroso. Non supporta la condizione di alta impedenza ('Z') né le condizioni indeterminate ('X').

I valori speciali di **std_logic** sono:

- **X** (indeterminato): rappresenta uno stato non definito dato da un conflitto tra due segnali;
- **Z** (alta impedenza): indica che la linea non è pilotata (tri-state);
- **H** e **L**: alta e bassa resistenza;
- **U** (uninitialized): segnale non inizializzato, usato solo nella simulazione;
- **W**: analogo di X per conflitti puramente resistivi;
- **-** (*don't care*): utilizzato nella sintesi logica per assegnare un'etichetta di irrilevanza al valore logico che la funzione può assumere in corrispondenza di specifici input, utile per ottimizzare il costo della sintesi (ad esempio nella copertura delle mappe di Karnaugh).

Nel contesto di VHDL, i **wires** sono utilizzati per trasmettere segnali singoli, mentre i **bus** trasmettono più segnali contemporaneamente. Per le costanti, si usano le virgolette singole per un wire (`my_wire <= '1';`) e le virgolette doppie per un bus di tipo `std_logic_vector` (`my_bus <= "11001010";`). Il tipo `std_logic_vector` è un array di elementi di tipo `std_logic`: ad esempio, `std_logic_vector(7 downto 0)` rappresenta un bus di 8 bit. La sintassi `downto` viene utilizzata per definire un vettore in cui il bit più significativo si trova all'indice più alto (`signal a: std_logic_vector(7 downto 0); a <= "00000001";` pone 1 in posizione meno significativa, cioè $a = 1$), mentre `to` è l'opposto, con il bit più significativo all'indice più basso (`signal a: std_logic_vector(0 to 7); a <= "00000001";` pone 1 in posizione più significativa, cioè $a = 128$). L'operatore di concatenazione, rappresentato da `&`, viene utilizzato per unire due o più segnali o vettori in un unico vettore più grande.

5.6 Signal vs Variable

Le **variabili** in VHDL hanno una semantica simile a quella dei linguaggi di programmazione come il C: servono a memorizzare valori utilizzati per l'elaborazione, ma è importante notare che *non producono hardware*. I **segnali**, al contrario, trasmettono informazioni circuitali e producono hardware, creando un registro fisico che conserva informazioni e generando circuiti reali. Questa distinzione è fondamentale, poiché variabili e segnali vengono utilizzati in modi diversi per rappresentare e gestire le informazioni nel design.

5.7 Statement Sequenziali e Concorrenziali

Le istruzioni **sequenziali** specificano l'ordine in cui devono essere eseguiti i passaggi di un algoritmo, funzionando come in un linguaggio di programmazione tradizionale dove l'ordine delle operazioni è cruciale: esse possono trovarsi solo all'interno di un **process**. Le istruzioni **concorrenti**, invece, descrivono la struttura di una porzione di circuito e specificano elaborazioni hardware che evolvono simultaneamente, senza richiedere un ordine specifico di esecuzione: i segnali e le connessioni tra i vari componenti vengono aggiornati in modo concorrente.

5.8 Esempi Comparati di Codice

Per comprendere come i linguaggi HDL descrivano l'hardware, è utile analizzare un esempio concreto scritto sia in SystemVerilog che in VHDL. Consideriamo la funzione $Y = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}C$, una somma di prodotti che fisicamente corrisponde a tre NOT, tre AND a tre ingressi e un OR a tre ingressi: un circuito combinatorio puro, senza alcun ordine di esecuzione, memoria o clock.

```
module aFunction (input logic a, b, c,
                  output logic y);
```



```

    assign y = ~a & ~b & ~c |
               a & ~b & ~c |
               a & ~b & c;
endmodule

```

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity aFunction is
    port (a, b, c : in STD_LOGIC;
          y : out STD_LOGIC);
end;
architecture behavior of aFunction is
begin
    y <= ((not a) and (not b) and (not c)) or
         (a and (not b) and (not c)) or
         (a and (not b) and c);
end;

```

Il codice VHDL è suddiviso in tre parti: la prima importa la libreria IEEE STD_LOGIC_1164, necessaria per il tipo STD_LOGIC; segue la **entity**, che definisce l'interfaccia (i tre ingressi e l'uscita); e infine la **architecture**, che descrive come funziona il circuito tramite un'assegnazione concorrente. Il risultato è una rete combinatoria che implementa fisicamente la funzione richiesta: le parentesi sono necessarie perché in VHDL gli operatori logici non hanno precedenze implicite.

Un secondo esempio riguarda un addizionatore a 32 bit. In VHDL, la stessa operazione è descritta tramite un'entity con ingressi e uscita come STD_LOGIC_VECTOR(31 downto 0), con l'assegnazione `y <= a + b;`. È importante sottolineare che il simbolo `+` non indica un'operazione eseguita nel tempo, ma una rete hardware che realizza la somma binaria dei due vettori: il sintetizzatore trasformerà questa espressione in una rete di full adder, collegati in cascata o secondo un'architettura più efficiente come il carry-lookahead, dettaglio che rimane nascosto al progettista. L'uso delle librerie STD_LOGIC_1164 e STD_LOGIC_UNSIGNED è necessario in VHDL per permettere di trattare i vettori di bit come numeri su cui applicare l'operazione di addizione.

Una volta scritto un circuito, il primo passo non è costruirlo fisicamente ma **simularlo**, per verificare che la descrizione produca esattamente il comportamento previsto dalle specifiche. Il passo successivo è la **sintesi**: si distingue tra *architectural synthesis*, che traduce una descrizione comportamentale in una rete di blocchi logici, e *logic synthesis*, che converte il codice HDL in una **netlist** esplicita di tutte le porte logiche e delle loro connessioni, applicando ottimizzazioni per ridurre il numero di porte, il consumo di area e potenza, o il ritardo di propagazione.

5.9 Tipi Sintetizzabili

In VHDL, non tutte le parti di un progetto ammettono sintesi: la sintesi è limitata a un sottoinsieme specifico dei tipi e delle strutture del linguaggio. I tipi che ammettono sintesi comprendono i tipi enumerati come **bit** (due valori logici) e **boolean** (true/false), **std_logic** e **std_ulogic**, **character** (usato raramente ma supportato), e i tipi numerici come **integer**, **natural** e **positive**. Gli array ammettono sintesi se hanno confini statici definiti, come un vettore **std_logic_vector(7 downto 0)**, e i sottotipi sono ammessi se il loro range è un sottoinsieme di valori di tipo enumerato (ad esempio **subtype my_bit is std_logic range '0' to '1'**). Non tutti i tipi possono essere tradotti in hardware: ad esempio l'**Access Type**, cioè i puntatori a memoria, e il tipo **File**, sono adatti solo alla simulazione.

5.10 Logica Combinatoria e Operatori Bitwise

Nei sistemi digitali, la logica combinatoria è costituita da circuiti in cui l'uscita dipende esclusivamente dai valori degli ingressi nello stesso istante di tempo, senza memoria né stato interno: esattamente il comportamento delle porte logiche fisiche. Gli operatori bitwise agiscono su singoli bit o su interi bus di bit, generando reti di porte logiche che operano in parallelo: se **a** è un bus a 4 bit, ogni operazione bitwise viene eseguita in parallelo sui 4 bit, generando 4 porte logiche indipendenti (ad esempio `assign y = ~a;` in SystemVerilog o `y <= not a;` in VHDL creano una banca di 4 invertitori). Oltre al NOT, i principali operatori bitwise sono riassunti nella tabella seguente:

Operatore	SystemVerilog	VHDL	Significato
AND	<code>a & b</code>	<code>a and b</code>	AND bit per bit
OR	<code>a b</code>	<code>a or b</code>	OR bit per bit
XOR	<code>a ^ b</code>	<code>a xor b</code>	XOR bit per bit
NAND	<code>~(a & b)</code>	<code>a nand b</code>	NOT-AND
NOR	<code>~(a b)</code>	<code>a nor b</code>	NOT-OR

Se **a** e **b** sono bus di 4 bit, ogni operatore genera 4 porte logiche in parallelo, una per ogni coppia di bit: ad esempio l'istruzione `assign y1 = a & b;` genera quattro porte AND che operano simultaneamente sui bit di **a** e **b**. In SystemVerilog le istruzioni del tipo `assign y = a & b;` sono chiamate *continuous assignments*.

5.11 Conditional Assignment e Multiplexer

L'assegnazione condizionale permette di descrivere un multiplexer in modo compatto. In VHDL si utilizza il costrutto `when...else`:

```
y <= a when s = '0' else b;
```

che descrive un multiplexer 2 a 1: se il segnale di selezione **s** vale '0' l'uscita **y** segue **a**, altrimenti segue **b**. Il costrutto equivalente in SystemVerilog è l'operatore ternario `assign y = s ? b : a;`. Entrambe le forme descrivono la stessa rete combinatoria: un multiplexer realizzato tramite porte AND, OR e NOT (o tramite pass-transistor a livello fisico).

5.12 Segnali Interni e il Full Adder

Il **full adder** è un circuito combinatorio che somma tre bit (**a**, **b** e **cin**) producendo un bit di somma **s** e un riporto **cout**. È un blocco fondamentale dell'ALU e viene usato per costruire addizionatori multi-bit. Nel full adder si introducono i segnali *propagate* $p = a \oplus b$ e *generate* $g = a \wedge b$: **p** indica se il carry in ingresso viene propagato, mentre **g** indica se la coppia di bit genera direttamente un carry. Le equazioni sono $s = p \oplus cin$ e $cout = g \vee (p \wedge cin)$.

```
module fulladder (input logic a, b, cin,
                  output logic s, cout);
    logic p, g;
    always_comb begin
        p = a ^ b;           // blocking
        g = a & b;           // blocking
        s = p ^ cin;         // blocking
        cout = g | (p & cin); // blocking
    end
endmodule
```

```

architecture behavioral of fulladder is
begin
  process(a, b, cin)
    variable p, g : STD_LOGIC;
  begin
    p := a xor b;           -- blocking (variabile)
    g := a and b;           -- blocking (variabile)
    s <= p xor cin;         -- non blocking (segnale)
    cout <= g or (p and cin); -- non blocking (segnale)
  end process;
end;

```

La sensitivity list del processo deve includere **a**, **b** e **cin**, perché la logica combinatoria deve rispondere ai cambiamenti di qualunque ingresso: se ne mancasse uno, il codice potrebbe sintetizzare logica sequenziale, oppure comportarsi diversamente tra simulazione e sintesi. L'esempio usa assegnamenti *blocking* per *p* e *g*, così che ottengano i nuovi valori prima di essere usati per calcolare *s* e *cout* che dipendono da loro; poiché compaiono a sinistra dell'operatore `:=` dentro un `process`, *p* e *g* devono essere dichiarate come **variable**.

5.13 Precedenza degli Operatori

A differenza di linguaggi come C, in VHDL gli operatori logici (**and**, **or**, **not**, **xor**) non hanno una precedenza implicita ben definita e differiscono da un compilatore all'altro nell'interpretazione di espressioni miste senza parentesi: è quindi buona pratica esplicitare sempre l'ordine di valutazione tramite parentesi, come visto nell'esempio della somma di prodotti, per evitare ambiguità e garantire che la sintesi produca esattamente la rete di porte desiderata.

5.14 Tri-State, Alta Impedenza e Valori Indeterminati

Quando si lavora con gli HDL non esistono solo i valori logici 0 e 1: per modellare correttamente ciò che accade nei circuiti reali esistono valori speciali che rappresentano situazioni fisicamente problematiche o non determinate. Il valore **Z** indica alta impedenza, cioè un filo che non è guidato da nessuno; il valore **X** indica un valore logico non valido o indefinito. La X viene generata quando un segnale si trova in una situazione in cui il simulatore non può stabilire se il valore corretto sia 0 o 1: un caso tipico è quando due dispositivi tri-state cercano di guidare contemporaneamente lo stesso bus verso valori opposti (un cortocircuito logico chiamato *contenzione*), oppure quando un ingresso di una porta logica è in stato Z (flottante), rendendo l'uscita indefinita.

All'inizio della simulazione c'è un'altra sorgente di incertezza: i flip-flop non sono ancora stati inizializzati. Per questo in SystemVerilog i loro output partono in stato X, mentre in VHDL partono nello stato **U** (*uninitialized*). Questi valori servono a individuare bug: se un segnale viene usato prima di essere correttamente inizializzato, l'indeterminazione si propaga e compare nell'uscita. In SystemVerilog un segnale può assumere quattro valori (0, 1, Z, X); in VHDL, usando **STD_LOGIC**, i valori sono più ricchi ('0', '1', 'Z', 'X' e 'U'), con la differenza che 'U' indica esplicitamente un segnale mai inizializzato mentre 'X' indica un valore logicamente inconsistente o invalido. Le porte logiche hanno tabelle di verità estese che tengono conto di questi valori speciali: una porta AND restituisce sempre 0 se uno degli ingressi è 0, anche se l'altro è Z o X, perché fisicamente l'uscita è forzata a 0; in tutte le altre combinazioni che coinvolgono Z, X o U, l'uscita diventa X oppure U, perché il comportamento reale sarebbe imprevedibile. Quando in simulazione compaiono X o U, quasi sempre significa che c'è un errore nel progetto (un segnale non inizializzato, un bus lasciato flottante, o più dispositivi che guidano lo stesso filo), rendendo questi valori uno strumento di debug potentissimo.

Il buffer **tri-state** consente tre stati distinti (0, 1, alta impedenza), essenziale per condividere bus tra più dispositivi garantendo che solo uno alla volta possa trasmettere. Un esempio classico è la realizzazione strutturale di un multiplexer con due tri-state buffer che guidano lo stesso bus di uscita, di cui solo uno è attivo alla volta (uno quando $s = 0$, l'altro quando $s = 1$):

```
module mux2 (input logic [3:0] d0, d1,
             input logic s,
             output tri [3:0] y);
    tristate t0 (d0, ~s, y);
    tristate t1 (d1, s, y);
endmodule
```

Qui y è dichiarata come **tri** perché ha due possibili driver (i due buffer tri-state), e occorre risolvere il conflitto tra i due assegnamenti concorrenti allo stesso net.

5.15 Bit Coalescing e Output Splitting

Quando si progettano circuiti digitali è spesso necessario prendere singoli bit o sottoparti di bus e combinarli in un bus più grande: questa operazione si chiama **bit coalescing** (o *bit swizzling*). Dal punto di vista hardware non crea logica: non genera porte né operazioni aritmetiche, ma descrive solo come i fili vengono collegati.

```
assign y = {c[2:1], {3{d[0]}}, c[0], 3'b101};
```

Le parentesi graffe in SystemVerilog servono a concatenare bus e bit: questa riga costruisce y unendo, nell'ordine, i bit 2 e 1 di c , tre repliche del bit $d[0]$ (il costrutto $\{3\{d[0]\}$ significa proprio “ripeti tre volte”), il bit 0 di c e una costante binaria a 3 bit, per un totale di $2 + 3 + 1 + 3 = 9$ bit. In VHDL lo stesso concetto si esprime con l'operatore di concatenazione $\&$, che non va confuso con l'AND:

```
y <= c(2 downto 1) &
     d(0) & d(0) & d(0) &
     c(0) &
     "101";
```

Specificare correttamente la dimensione delle costanti è fondamentale: nella costante $3'b101$ il numero 3 dice che quella costante è larga esattamente 3 bit; se non fosse specificato, il tool potrebbe inserire zeri impliciti e creare un bus di dimensione sbagliata, spostando i bit nella posizione errata. Questo meccanismo è usato ovunque nei datapath, nei registri, negli indirizzi, nei formati delle istruzioni e nei pacchetti di comunicazione.

L'**output splitting** è l'operazione opposta: si prende un bus grande e lo si divide in più segnali. Se si moltiplicano due numeri a 8 bit, il prodotto richiede fino a 16 bit; spesso interessa separare la parte alta (*most significant byte*) dalla parte bassa (*least significant byte*), ad esempio per gestire l'overflow o per applicazioni DSP:

```
module mul (input logic [7:0] a, b,
            output logic [7:0] upper, lower);
    assign {upper, lower} = a * b;
endmodule
```

```
signal prod : STD_LOGIC_VECTOR(15 downto 0);
prod <= a * b;
upper <= prod(15 downto 8);
lower <= prod(7 downto 0);
```

La moltiplicazione $a \cdot b$ sintetizza un moltiplicatore combinatorio, mentre lo splitting non crea logica complessa: è cablaggio puro, cioè selezione di linee che porta i bit [15 : 8] a **upper** e i bit [7 : 0] a **lower**.

5.16 Sign Extension

Quando un numero con segno è rappresentato in binario (complemento a due), il bit più significativo è il *bit di segno*. Portare questo numero in un bus più grande (ad esempio da 16 a 32 bit) non si ottiene semplicemente aggiungendo zeri a sinistra, perché questo cambierebbe il valore dei numeri negativi: occorre invece copiare il bit di segno nelle nuove posizioni più significative, un'operazione chiamata **sign extension**.

```
assign y = {{16{a[15]}}, a[15:0]};
```

L'espressione {16{a[15]}} significa “ripeti 16 volte il bit a[15]”. In VHDL la stessa operazione viene descritta in modo più esplicito, controllando il bit di segno:

```
y <= X"0000" & a when a(15) = '0' else
      X"FFFF" & a;
```

Se il bit di segno è 0 si concatenano 16 zeri (zero-extension); se è 1 si concatenano 16 uno (sign-extension vera e propria). Dal punto di vista hardware, la sign extension non richiede calcoli: è solo una rete di fili che copia il bit di segno su tutti i bit più significativi. Questo meccanismo è essenziale nei processori, ad esempio quando un'istruzione carica un valore a 16 bit da memoria e deve usarlo in un'ALU a 32 bit, mantenendo corretto il segno del numero.

5.17 I Ritardi in Simulazione

Ogni porta logica reale ha un ritardo di propagazione: se un ingresso cambia, l'uscita non cambia istantaneamente, ma dopo un piccolo intervallo di tempo. Gli HDL permettono di modellare questi ritardi tramite il concetto di **delay**. In SystemVerilog si usa il simbolo # (**assign #1 bb = a;**), con le unità di tempo definite dalla direttiva **timescale** (ad esempio **timescale 1ns / 1ps;**); in VHDL si usa la clausola **after** (**bb <= not a after 1 ns;**). Il punto fondamentale è che questi delay *non vengono sintetizzati*: servono solo in simulazione per capire come i segnali si propagano, individuare glitch, studiare problemi di temporizzazione e trovare errori di progettazione. Quando il circuito viene sintetizzato, il tool ignora i delay e costruisce l'hardware in base alle porte e ai collegamenti, non ai valori numerici scritti dopo # o after.

5.18 Costrutti di Controllo Avanzati: Case, If-Else e Generate

Oltre agli assegnamenti concorrenti (**assign**), è possibile descrivere comportamenti anche usando blocchi **always** (SystemVerilog) o **process** (VHDL), purché la sensitivity list risponda a tutti gli ingressi responsabili dei cambiamenti dell'uscita: **always_comb** in SystemVerilog e **process(a, b, c, ...)** in VHDL descrivono così reti di porte, non sequenze di istruzioni software.

Un esempio classico è il **decoder a 7 segmenti**, che ad alto livello di astrazione traduce un numero a 4 bit nella corrispondente cifra decimale su un display:

```
always_comb
  case (data)
    0: segments = 7'b1111110;
    1: segments = 7'b0110000;
    // ...
    9: segments = 7'b1111011;
```

```
default: segments = 7'b0000000;
endcase
```

In un **case** dentro un blocco **always** devono essere coperti tutti i casi possibili: se mancano dei valori, l'uscita mantiene il valore precedente, generando un *latch* indesiderato invece di logica combinatoria pura. Per questo la clausola **default** (in VHDL, **when others**) è obbligatoria se si vuole vera logica combinatoria. Un **case** completo dentro **always_comb** o **process** descrive in sostanza una rete equivalente a una ROM o a una tabella di verità.

Gli statement **always/process** possono contenere anche istruzioni **if**: quando tutte le combinazioni degli ingressi sono gestite (if-elsif-else completo) si ottiene logica combinatoria, altrimenti logica sequenziale indesiderata. Un esempio tipico è il **priority encoder**, che restituisce l'ingresso più significativo che vale 1:

```
always_comb
if      (a[3]) y = 4'b1000;
else if (a[2]) y = 4'b0100;
else if (a[1]) y = 4'b0010;
else if (a[0]) y = 4'b0001;
else      y = 4'b0000;
```

SystemVerilog fornisce anche lo statement **casez**, utile per descrivere tabelle di verità con “don't care” (indicati con ?), come nella riscrittura dello stesso circuito di priorità:

```
always_comb
casez(a)
4'b1??? : y = 4'b1000;
4'b01?? : y = 4'b0100;
4'b001? : y = 4'b0010;
4'b0001 : y = 4'b0001;
default : y = 4'b0000;
endcase
```

qui 1??? significa “a[3]=1, gli altri bit non contano”. If/else e case/casez sono a tutti gli effetti selettori hardware: un if a priorità descrive un circuito di priorità fisico, non una decisione software eseguita nel tempo. Vale una regola pratica sui blocchi **always**: per la logica sequenziale (registri, flip-flop) si usa **always_ff @(posedge clk)** con assegnamenti non-blocking (**<=**); per la logica combinatoria semplice si usano assegnamenti concorrenti (**assign**); per logica combinatoria più complessa si usa **always_comb** con assegnamenti blocking (**=**); infine, un segnale deve avere un solo driver (a eccezione dei bus tri-state), pena un conflitto che genera hardware sbagliato.

Un ultimo costrutto utile per la sintesi parametrica è il **generate statement**, che permette al tool di creare un numero variabile di istanze o porte in base a un parametro, senza dover scrivere manualmente N righe di codice: ad esempio, per costruire una catena di porte AND a 2 ingressi che propagano un segnale ($x_1 = a_0 \wedge a_1$, $x_2 = x_1 \wedge a_2$, ...), un ciclo **for-generate** permette di dire al compilatore “ripeti questa struttura per $i = 1 \dots N - 1$ ”. È importante sottolineare che il **generate** non è un costrutto a runtime: la sua elaborazione avviene a tempo di compilazione/sintesi, ed è in quel momento che si decide quanta circuiteria fisica viene effettivamente generata. Un caso applicativo tipico è il decoder generico $N \rightarrow 2^N$ one-hot, dove un singolo assegnamento $y[a] = 1$; (con tutte le altre uscite azzerate) genera automaticamente 2^N linee di uscita, qualunque sia N ; in VHDL, poiché gli indici di un vettore devono essere di tipo **integer** e non **STD_LOGIC_VECTOR**, sono necessarie conversioni esplicite tramite le funzioni **CONV_INTEGER** e **CONV_STD_LOGIC_VECTOR**.

Nota su SystemVerilog: quando si usano vettori multidimensionali, è utile distinguere tra **packed array**, un unico vettore di bit contiguo suddiviso logicamente in più campi (ad esempio `logic [2:0][5:0] a1;`, ideale per registri suddivisi in campi, istruzioni di CPU o pacchetti di

dati, poiché solo i packed array sono garantiti contigui in bit e quindi facilmente concatenabili o reinterpretabili come unico valore), e **unpacked array**, un vero array di elementi separati e non necessariamente contigui (`logic [5:0] a2 [2:0];`, più adatto a banchi di registri o memorie). VHDL non prevede questa distinzione, poiché tratta in modo nativo array di segnali indipendenti.

5.19 Logica Sequenziale in VHDL

5.19.1 Reset Sincrono e Asincrono

Un registro con reset sincrono aggiorna il proprio stato solo sul fronte di clock, indipendentemente dal segnale di reset, che viene quindi campionato come un normale ingresso: se `reset='1'` al momento del fronte di clock, il registro si azzerava, altrimenti carica il nuovo valore. Un registro con reset asincrono, al contrario, ha il reset nella sensitivity list assieme al clock, e agisce immediatamente non appena `reset` si attiva, senza attendere il fronte di clock: questo garantisce un'inizializzazione immediata del sistema mediante un segnale esterno indipendente dal clock, ma introduce il rischio di problemi di sincronizzazione (metastabilità alla rimozione del reset) se non gestito con attenzione.

5.19.2 Registro con Enable

Un registro con segnale di `enable` aggiorna il proprio valore sul fronte di clock solo se l'enable è attivo; in caso contrario mantiene il valore precedente. Descrittivamente, questo corrisponde a un multiplexer posto davanti a un normale registro, che sceglie tra il nuovo valore in ingresso e il valore attualmente memorizzato in retroazione.

5.19.3 Latch Trasparenti

Il **D Latch** è un elemento di memoria di livello, sensibile non al fronte ma al *livello* del clock: quando il clock è alto (attivo), il latch è trasparente e l'ingresso passa direttamente all'uscita; quando il clock è basso, il valore viene mantenuto (l'uscita "congela" l'ultimo dato visto). Questo comportamento è molto diverso da un flip-flop, che campiona solo sul fronte di clock. In VHDL:

```
process(clk, d)
begin
    if clk = '1' then
        q <= d;
    end if;
end process;
```

Se non necessario, è preferibile usare flip-flop edge-triggered invece dei latch, perché questi ultimi creano cammini temporali difficili da controllare e possono introdurre race condition: un `if` senza `else` in logica sequenziale genera infatti un latch. Questo è un errore molto comune e insidioso: in SystemVerilog e VHDL, un `if` incompleto all'interno di un processo sequenziale, invece di generare l'errore atteso, sintetizza silenziosamente un latch trasparente.

5.20 Organizzazione della Memoria e Decodifica degli Indirizzi

Nel modello ideale, un microprocessore utilizza un bus di indirizzi $A[0 : N - 1]$ per accedere a una memoria unica contenente 2^N locazioni: ogni combinazione dei bit di indirizzo identifica una cella di memoria. In pratica, però, una memoria così grande non viene realizzata come un unico chip, ma come un insieme di moduli più piccoli: una memoria da 2^N locazioni può essere costruita usando due memorie da 2^{N-1} locazioni ciascuna. Il bus degli indirizzi viene

allora diviso in due parti: i bit meno significativi $A[0 : N - 2]$ vengono inviati a entrambi i chip e selezionano la cella interna al singolo banco, mentre il bit più significativo $A[N - 1]$ non seleziona una cella, ma viene usato per scegliere quale dei due chip deve essere attivo, portato a un circuito di decodifica che genera due segnali di enable. Questo meccanismo può essere esteso a più banchi utilizzando più bit dell'indirizzo, con la logica di decodifica che diventa tanto più complessa quanto più piccolo è ciascun banco.

5.21 Divide-by-3 FSM: un Esempio Completo

Un esempio classico e istruttivo è una FSM che produce in uscita un segnale y che vale 1 ogni tre cicli di clock, un divisore di frequenza per 3, usando tre stati che rappresentano il resto della divisione per 3: S_0 (resto 0), S_1 (resto 1), S_2 (resto 2). Ad ogni clock la FSM avanza secondo il ciclo $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_0 \rightarrow \dots$.

```
-- Registro di stato
process(clk)
begin
    if clk'event and clk = '1' then
        if reset = '1' then state <= "00";
        else state <= nextstate;
        end if;
    end if;
end process;

-- Logica di next-state
nextstate <= "01" when state = "00" else
             "10" when state = "01" else
             "00";

-- Logica di uscita (Moore FSM: dipende solo dallo stato)
y <= '1' when state = "00" else '0';
```

Dato che S_0 arriva ogni tre cicli di clock, y è alto una volta ogni tre cicli, realizzando così la divisione per 3.

5.22 Macchine a Stati Finiti: Moore e Mealy

Una **FSM** (Finite State Machine) è un modello di circuito digitale che combina logica sequenziale e logica combinatoria per descrivere un sistema che può trovarsi solo in un numero finito di stati. Ogni FSM è composta da tre blocchi fondamentali: lo **state register**, che memorizza lo stato attuale e cambia solo sul fronte di clock; la **next-state logic**, una rete combinatoria che calcola il prossimo stato in funzione dello stato presente e degli ingressi; e la **output logic**, che genera le uscite della FSM.

Esistono due grandi famiglie di FSM. In una **Moore FSM**, le uscite dipendono solo dallo stato presente ($output = f(\text{stato})$), il che significa che gli output cambiano solo quando cambia lo stato, e quindi solo sul fronte di clock: questo modello garantisce uscite molto stabili e meno rischio di glitch, ed è il più usato nei sistemi sincroni. In una **Mealy FSM**, invece, le uscite dipendono sia dallo stato presente sia dagli ingressi ($output = f(\text{stato}, \text{input})$): se cambia un ingresso, può cambiare subito anche l'uscita senza aspettare il prossimo clock, rendendo la macchina più reattiva ma esponendola al rischio di glitch se gli ingressi oscillano.

Un esempio più elaborato di FSM di tipo Mealy è quello di una macchina con un ingresso a e un'uscita che vale 1 quando l'ingresso attuale è uguale a quello che era nei due cicli di clock precedenti: qui la FSM deve ricordare, attraverso i suoi stati, la storia recente dell'ingresso, e l'uscita dipende sia dallo stato (cioè da cosa è successo nei cicli passati) sia dall'ingresso

corrente. Per leggibilità e per evitare errori, è buona pratica usare l'enumerazione degli stati (in VHDL, `type statetype is (S0, S1, S2);`) invece di numeri binari grezzi, lasciando al tool di sintesi la scelta dell'encoding.

5.23 Type Idiosyncrasies in VHDL

A differenza di SystemVerilog, VHDL impone un sistema di tipi molto rigido, che protegge l'utente da alcuni errori ma rende anche il linguaggio più verboso. `STD_LOGIC` e `STD_LOGIC_VECTOR` non hanno operazioni aritmetiche native (addizione, confronto, shift, conversione a interi), definite invece nelle librerie `IEEE.NUMERIC_STD` e `IEEE.STD_LOGIC_SIGNED`. VHDL ha anche un tipo `BOOLEAN` con valori `true` e `false`: è facile confondersi pensando che `true` sia equivalente a `STD_LOGIC = '1'`, ma questi tipi non sono intercambiabili, e bisogna sempre scrivere il confronto esplicito `s = '1'` anziché usare direttamente `s`. VHDL ha inoltre un tipo `INTEGER`, usato come indice dei bus: non si può indicizzare direttamente un bus con uno `STD_LOGIC_VECTOR`, ma bisogna convertirlo in `INTEGER` tramite la funzione `CONV_INTEGER`, definita nella libreria `STD_LOGIC_UNSIGNED`.

Un principio importante nella progettazione è che un'uscita non è un normale segnale interno, ma rappresenta fisicamente un pin del chip pilotato da un buffer di uscita: non è un nodo logico “libero” su cui si possono fare calcoli, ma il punto finale della catena. Per questo motivo, se un valore deve essere usato sia per il calcolo interno sia come uscita, non si deve mai usare direttamente l'uscita come variabile intermedia, ma introdurre un segnale interno che rappresenta il risultato logico e poi collegarlo all'uscita. Le uscite vanno *pilotate*, non lette.

5.24 Moduli Parametrizzati (Generic)

In VHDL, il concetto di modulo parametrizzato si chiama **generic**: si dichiara `generic (N: integer := 8);` e poi si usano le porte con `STD_LOGIC_VECTOR(N-1 downto 0)`. Questo permette di scrivere una volta sola un modulo, come un multiplexer, e scegliere la larghezza N quando lo si istanzia (`generic map (N => 12)`), fondamentale per riusare lo stesso blocco su bus di dimensioni diverse in progetti reali come datapath, registri e indirizzi.

5.25 Memorie in HDL

Nella progettazione di memorie in VHDL si distinguono diverse architetture. Le **RAM con bus separati** hanno un bus `din` (write data) e un bus `dout` (read data) distinti, con scrittura sincrona quando il write enable è attivo. Le **RAM con bus multiplexato** usano invece un unico bus dati bidirezionale (`inout`): quando si scrive, la CPU guida il bus e la RAM legge; quando si legge, la RAM guida il bus e la CPU legge; per evitare conflitti, quando nessuno guida il bus, esso va in stato Z. Nel VLSI moderno e su FPGA si preferiscono spesso connessioni point-to-point e multiplexer interni, poiché i tri-state “globali” non sono sempre supportati come un tempo, ma resta un concetto didattico fondamentale per capire la risoluzione dei net e i bus condivisi.

I **register file multiport** permettono più letture e scritture simultanee nello stesso ciclo (ad esempio due porte di lettura per alimentare un'ALU e una porta di scrittura per il risultato), e sono la base dei datapath dei microprocessori. Le **ROM**, infine, sono spesso descritte tramite un semplice `case`, che il sintetizzatore trasforma in logica combinatoria (reti di multiplexer/porte) o in una macro ROM/lookup-table a seconda della dimensione: per ROM piccole conviene la logica, per ROM grandi conviene la macro dedicata.

5.26 Testbench

Un **testbench** è un modulo HDL che istanzia il *device under test* (DUT), genera stimoli e osserva le uscite. Il testbench più semplice applica combinazioni una dopo l'altra con ritardi temporali, e si verifica il comportamento guardando le waveform: è utile per esempi piccoli ma non scala bene su progetti complessi. Un salto di qualità si ha con la **verifica automatica**, in cui il testbench conosce l'output atteso e usa costrutti di **assert** per giudicarsi da solo (pass/-fail) – in SystemVerilog `assert(...) else $error("...");`; in VHDL `assert condition report "..." severity error;` – permettendo di accorgersi immediatamente se qualcosa si rompe nel design. Quando i vettori di test diventano numerosi, conviene leggerli da un file esterno: il testbench apre il file all'inizio della simulazione, legge una riga alla volta, applica gli input al DUT, si sincronizza con il clock, confronta l'output reale con quello atteso e produce un report finale degli errori. In questo modo il testbench diventa “data-driven”: cambiando i test non si modifica il codice, ma solo il file di vettori.

Nota (approfondimento a livello di netlist, solo SystemVerilog): a un livello di descrizione ancora più basso, SystemVerilog permette di modellare circuiti tramite primitive di tipo transistor (`tranif1`, `tranif0`, ecc.), utili ad esempio per descrivere una porta NOR pseudo-nMOS (rete di pull-down a nMOS verso massa comandata dagli ingressi, con un pull-up debole verso V_{DD}) o un latch a livello transistor, dove per evitare che un nodo interno privo di driver “fluttui” si utilizza il tipo di rete `triereg`, che mantiene l'ultimo valore per carica residua. Queste primitive sono utili didatticamente per comprendere fenomeni di bus tri-state, contese e nodi flottanti, ma nella progettazione RTL moderna si preferisce descrivere il comportamento e lasciare alla tecnologia l'implementazione fisica, poiché la sintesi vera raramente opera a questo livello.

6 FPGA nel Dettaglio

6.1 Logiche Programmabili e il Trade-off Economico

In questo ambito, con logiche programmabili intendiamo non macchine di Turing, bensì logiche la cui configurazione circuitale è programmabile: queste logiche non possono eseguire dei programmi, ma la loro struttura circuitale è completamente configurabile. I microprocessori sono i dispositivi che consumano di più, mentre a parità di consumo i circuiti dedicati sono i più performanti, seppur con un costo importante; le FPGA hanno il vantaggio di essere poco costose e di avere un ottimo rapporto consumo/performance.

Riprendendo la distinzione tra costi fissi e variabili introdotta in precedenza, l'andamento nel tempo mostra come diminuiscano i costi di progettazione dell'hardware mentre aumentano le capacità tecnologiche, creando una finestra centrale di massima convenienza per i circuiti custom o semi-custom. Questa **finestra di opportunità** è strettamente legata al concetto di *time-to-market*: un ritardo nell'uscita di un prodotto riduce il profitto totale, poiché il picco delle vendite avviene comunque nello stesso istante temporale ma con un valore minore, essendo partite in ritardo, e una parte delle vendite viene persa per sempre poiché il mercato ha una finestra temporale limitata prima della fine del ciclo di vita del prodotto. Con l'evoluzione della tecnologia dei circuiti integrati, le prestazioni sono cresciute esponenzialmente, ma purtroppo anche i costi, in particolare i costi fissi di progetto e produzione: questo ha fatto sì che la finestra temporale ed economica in cui conviene realizzare circuiti custom si sia progressivamente ristretta, spingendo verso la strategia di ridurre i costi fissi anche a scapito di un leggero aumento dei costi per singolo pezzo.

Confrontando le tre tecnologie sul piano del costo al variare del volume prodotto, la **FPGA** parte con un costo molto basso per pochi pezzi (non ha costi di maschere né di fabbricazione dedicata: si compra il chip già fatto e lo si programma), ma ogni singolo chip è costoso, quindi

il costo cresce rapidamente con il volume. L'**MGA** (Gate Array) ha costi fissi medi e costi unitari più bassi degli FPGA, rappresentando una soluzione intermedia. Il **CBIC** (ASIC full custom o standard-cell) ha costi fissi enormi ma costo per pezzo molto basso, e conviene solo producendo volumi molto elevati.

6.2 Architettura di Sistema FPGA

L'architettura di un sistema FPGA è sempre la stessa: alla periferia del chip ci sono i blocchi di **I/O**, che acquisiscono un segnale analogico influenzando il segnale finale e possono fare sia da ingresso che da uscita; la parte centrale del chip è realizzata tramite un array regolare di logiche programmabili che, quando interconnesse, generano un certo andamento funzionale complessivo. È bene ricordare che i segnali logici sono una semplice astrazione: gli unici segnali che realmente esistono sono le forme d'onda analogiche che, opportunamente interpretate, “diventano” segnali logici digitali. Gli elementi costitutivi sono quindi gli I/O block e i blocchi logici, interconnettibili attraverso switch di interconnessione, creando così una matrice di blocchi programmabili interconnessi su cui si può programmare ciascun blocco per fare la funzione desiderata, e dato che risiedono su RAM è possibile riprogrammare i blocchi a piacere.

6.3 Programmabilità Fisica

Le **ROM** (read-only-memory) sono memorie a sola lettura, contrapposte alle **RAM** (random-access memory), nate come evoluzione della memoria magnetica: un tempo le memorie erano seriali, poi con le memorie a semiconduttore l'accesso è diventato random, permettendo di selezionare il bit che interessa senza dover scorrere sequenzialmente tutta la catena. Le ROM sono più o meno la stessa cosa ma non sono riprogrammabili né scrivibili come le RAM.

In origine, le interconnessioni non erano riprogrammabili, essendo create con processi irreversibili come il bruciare un fusibile: questo è il principio delle **PROM** (programmable read only memory), dove una volta scritto il dato con questo processo distruttivo non si poteva più tornare indietro. In seguito venne considerata un'alternativa reversibile e non volatile, realizzata con un MOSFET a doppio gate: le **EPROM** (erasable programmable read only memory), memorie cancellabili tramite esposizione a luce ultravioletta. Se nel gate intermedio non c'è carica, il MOSFET funziona normalmente; se il gate intermedio ha carica sufficiente per bilanciare quella del gate superiore, non si crea il canale di inversione e il MOSFET non funziona mai. Le **EEPROM** sono come le EPROM ma possono essere cancellate elettricamente, senza dover far uso dei raggi UV.

Un PLD contiene componenti sia di logica che di memoria (contenenti le informazioni di configurazione), che possono essere di tipo antifusibili al silicio, SRAM, Flash, o celle EPROM. Gli **antifusibili al silicio** sono originariamente normalmente OFF: la programmazione, una volta realizzata, non era più riconfigurabile. La procedura più utilizzata era l'*antifuse*, dove un plug resistivo (un fusibile) poteva essere fuso applicando una data tensione, mettendo definitivamente in contatto due piste e realizzando una connessione permanente. I circuiti che utilizzano questa tecnologia impiegano una barriera sottile di silicio amorfo tra due conduttori metallici: applicando una tensione sufficientemente alta (un breve impulso di circa un millisecondo dell'ampiezza di circa 16 volt), tutto il silicio amorfo si trasforma in una lega policristallina silicio-metallo con bassa resistenza, conduttiva.

Per il controllo non volatile tramite EPROM ed EEPROM, si tratta di una programmazione non distruttiva ma riprogrammabile e recuperabile: fa uso di un transistor MOS paragonabile a un “rubinetto”. Applicando una tensione di soglia sul gate si crea un canale di elettroni; isolando il gate con un ossido, il MOS diventa un condensatore con un'armatura completamente isolata. Sopra il gate viene posto un secondo gate flottante, e applicando una tensione maggiore della soglia sul gate di controllo, la carica si sposta verso il gate flottante, creando o eliminando

la connessione e programmando così il circuito. La programmazione è reversibile perché il processo non è distruttivo: per ripristinare le condizioni iniziali basta esporre il circuito a luce UV.

Il controllo tramite **Static RAM**, infine, si basa sulla programmazione della RAM stessa: nel momento in cui si spegne il circuito, la memoria “scompare” trattandosi essenzialmente di un flip-flop, ovvero una cella di memoria RAM statica, con due inverter retroazionati che fungono da latch di memoria.

6.4 Programmabilità Logica

La programmabilità logica si riferisce alla capacità di configurare dinamicamente il comportamento di un circuito logico, modificando le connessioni tra i blocchi logici interni per implementare diverse funzioni. Un esempio prototipico è l'**Actel ACT 1**, una delle prime implementazioni che ha sfruttato blocchi logici configurabili e un'architettura a matrice regolare; i multiplexer, utilizzati come blocchi di configurazione fondamentali, offrono la possibilità di selezionare diverse combinazioni di input, rendendo il sistema adattabile a varie applicazioni logiche.

La progettazione delle celle logiche programmabili mira a realizzare funzioni complesse attraverso una struttura modulare, che permette di suddividere funzioni logiche avanzate in blocchi più semplici, riprogrammabili e riutilizzabili in diverse combinazioni. Consideriamo, ad esempio, la funzione $F = AB + B'C + D$: essa può essere riformulata come $F = B(A + D) + B'(C + D) = B \cdot F_2 + B' \cdot F_1$, dove $F_1 = C + D$ e $F_2 = A + D$. Questa formulazione consente di implementare F suddividendola in due sotto-funzioni configurate su blocchi logici più semplici e poi interconnesse.

La versatilità della programmabilità logica può essere compresa esaminando il numero di funzioni realizzabili con due variabili binarie: con due ingressi ci sono $2^2 = 4$ configurazioni di input possibili, il che implica la possibilità di creare $2^4 = 16$ funzioni logiche diverse. Un singolo multiplexer può essere configurato per realizzare molte di queste funzioni fondamentali, collegando opportunamente i suoi ingressi dati per rappresentare tutte le combinazioni di funzione logica basate su due variabili: questa configurazione rappresenta una forma semplificata di programmabilità logica.

6.5 Blocchi Logici Configurabili (CLB)

Il **Configurable Logic Block** (CLB) rappresenta il componente fondamentale nelle FPGA: ogni CLB contiene elementi logici programmabili e risorse di memoria che possono essere configurati per implementare funzioni combinatorie o sequenziali. All'interno di ogni CLB sono tipicamente presenti blocchi logici combinatori realizzati con **Look-Up Table** (LUT) per implementare funzioni logiche, flip-flop per la memorizzazione di valori logici e la gestione di operazioni sequenziali, e multiplexer per selezionare gli input o combinare le uscite.

Definizione 6.1 (Look-Up Table). *Una LUT è essenzialmente una piccola memoria che memorizza i valori di output per tutte le combinazioni possibili di input. Configurando i bit della memoria interna, la LUT può essere programmata per realizzare qualsiasi funzione logica combinatoria, rendendo le LUT estremamente flessibili, efficienti per funzioni complesse e naturalmente adatte al parallelismo. La struttura è composta da ingressi logici che funzionano come indirizzo della memoria, una memoria interna che contiene i valori della funzione, e un'uscita che restituisce il valore memorizzato all'indirizzo selezionato.*

L'evoluzione storica delle celle logiche mostra bene questi concetti. Le celle **ACTEL ACT1** utilizzano un'architettura basata su antifusibili per interconnessioni permanenti: alte

prestazioni e basso consumo energetico, ma configurazione irreversibile. Il modello di temporizzazione è definito dal percorso critico $t_{PD} + t_{SUD} + t_{CO}$, dove t_{PD} è il tempo di propagazione (dipendente dalla funzione combinatoria implementata), t_{SUD} è il tempo di setup, t_{CO} è il tempo di clock-to-output (influenzato dal fan-out), e t_H è il tempo di hold del flip-flop. Le celle **Xilinx XC3000** utilizzano un'architettura basata su LUT a 3 ingressi per implementare funzioni logiche combinatorie, offrendo una maggiore flessibilità rispetto agli antifusibili grazie alla riprogrammabilità. Le celle **Xilinx XC4000**, evoluzione delle precedenti, offrono una LUT a 4 ingressi, consentendo funzioni logiche più complesse.

6.6 Architettura Xilinx Spartan-II

L'FPGA **Xilinx Spartan-II** è organizzato come una grande matrice di CLB al centro del chip, collegati da una rete di interconnessioni configurabili, e affiancati da blocchi specializzati che svolgono funzioni dedicate: ai lati sono presenti le **Block RAM**, che forniscono vere memorie hardware efficienti per dati e buffer, mentre lungo il perimetro si trovano i blocchi di I/O che gestiscono l'interfacciamento elettrico con l'esterno; agli angoli sono invece collocati i **DLL** (Delay Locked Loop), che servono a distribuire e sincronizzare correttamente il clock riducendo lo skew. Insieme, questi elementi permettono all'FPGA di implementare sistemi digitali completi, in cui la logica combinatoria e sequenziale viene realizzata nei CLB, la memoria nei blocchi RAM dedicati, il timing nei DLL e la comunicazione con l'esterno nei moduli di I/O.

6.7 Sintesi RTL Tradizionale vs High-Level Synthesis

Nel flusso di **RTL synthesis** tradizionale, il progetto hardware viene descritto direttamente in VHDL o Verilog a livello di registri e segnali; il codice viene prima verificato tramite simulazione RTL, poi sintetizzato, mappato sull'hardware (place & route) e infine testato sul sistema reale. Questo processo è fortemente iterativo: se dopo il place & route non si rispettano i vincoli di timing, area o potenza, si deve tornare indietro a modificare il codice RTL, e questo ciclo di *design closure* richiede personale altamente specializzato e tempi molto lunghi, potendo arrivare facilmente a decine o centinaia di persone-mese nei progetti industriali.

Con la **High-Level Synthesis** (HLS), invece, il progettista scrive l'algoritmo in C, C++ o SystemC, un linguaggio ad alto livello molto più compatto ed espressivo. Il tool di HLS traduce automaticamente questo codice in RTL, generando un blocco hardware IP (Intellectual Property), che può essere inserito in una libreria di IP, cioè blocchi hardware già pronti che si riutilizzano e si collegano come moduli, in un'analogia diretta all'uso di funzioni di libreria in C. Anche qui esiste una retroazione di debug, ma avviene a livello di C e di sistema, molto più velocemente rispetto all'RTL: per questo il flusso HLS può ridurre drasticamente i tempi di sviluppo, arrivando anche a essere dieci-quindici volte più veloce rispetto al flusso RTL tradizionale.

6.8 Interconnessioni e Ritardi

Le interconnessioni rappresentano una componente cruciale nelle FPGA, poiché determinano la capacità di collegare tra loro i CLB e di definire il comportamento complessivo del circuito. Tuttavia richiedono un elevato utilizzo di risorse, in particolare moduli RAM per gestirne la programmazione, e costituiscono la principale causa di ritardo nel circuito, influenzando in modo significativo le prestazioni generali. Il *channel* rappresenta l'area dedicata alle linee di interconnessione tra i CLB, composta da più piste e punti di intersezione che permettono la programmazione delle connessioni; la matrice di interconnessione utilizza un pass-transistor tra ogni possibile coppia di linee, e attivando selettivamente i pass-transistor è possibile stabilire quali collegamenti sono attivi.

La stima del ritardo di propagazione, il *critical path*, è un'analisi fondamentale per verificare che i ritardi siano compatibili con i requisiti temporali dell'applicazione, e dipende sia dalla funzione implementata sia dalla sua disposizione fisica: per questo è necessaria una verifica post-layout. Il **ritardo di Elmore** è un modello utilizzato per stimare i tempi di propagazione nelle interconnessioni, basato sull'analisi di resistenza e capacità delle linee di connessione. Nel caso delle FPGA Actel, il ritardo è fortemente influenzato dalla semplicità dell'architettura basata su antifusibili, che riduce il carico parassita e quindi i ritardi complessivi. L'architettura Xilinx, invece, si basa su una LUT a 5 ingressi riconfigurabile come due LUT a 4 ingressi: questa flessibilità comporta un aumento del ritardo di propagazione rispetto alle interconnessioni basate su antifusibili.

6.9 Condizionamento del Segnale e Blocchi I/O

Il condizionamento del segnale è un processo fondamentale nel trattamento dei segnali analogici, per consentire la loro corretta interpretazione come segnali digitali: poiché i segnali analogici possono avere andamenti variabili e non sempre interpretabili univocamente, è necessario normalizzarli e filtrarli. I blocchi di I/O gestiscono l'interazione tra l'esterno del chip e i circuiti interni, svolgendo funzioni di condizionamento dei segnali esterni, protezione contro scariche elettrostatiche e fornitura di alimentazione e riferimenti di tensione. Ogni dispositivo che pilota una linea esterna è considerato un *buffer*.

La configurazione **totem-pole** è uno stadio attivo in cui due transistor lavorano in opposizione di fase (uno acceso, l'altro spento), includendo diodi di protezione o di clamping che proteggono da sovratensioni o sottotensioni dovute a carichi induttivi: se una bobina accumula energia induttiva e il generatore viene spento, la bobina tende a richiamare corrente causando un aumento di tensione pericoloso, e i diodi di protezione evitano che il dielettrico tra drain e gate dei transistor venga danneggiato. Il buffer tri-state consente tre stati distinti (0, 1, alta impedenza), essenziale per condividere bus tra più dispositivi garantendo che solo uno alla volta possa trasmettere.

In presenza di commutazioni rapide rispetto alle impedenze coinvolte, si deve considerare il tempo di volo t_f , tipicamente dell'ordine di 1 ns ogni 30 cm di linea di trasmissione: gli effetti di propagazione diventano significativi quando la commutazione avviene in un tempo minore di due volte il tempo di volo, generando fluttuazioni indesiderate nel segnale. Per mitigare questi problemi si utilizzano terminazioni di adattamento, come il circuito aperto, la resistenza in parallelo, la terminazione di Thévenin, l'adattamento alla sorgente e l'adattamento in parallelo con condensatore in serie.

Per prevenire errori di interpretazione dovuti al fenomeno dell'*input bouncing*, si ricorre a tecniche di **debouncing**. Il debouncing con flip-flop SR implementa un circuito anti-rimbalzo tramite porte NAND o NOR, memorizzando lo stato dell'uscita e ignorando gli impulsi di disturbo. Il debouncing con Trigger di Schmitt introduce un'isteresi che trasforma un segnale analogico rumoroso in uno digitale pulito, con l'uscita che varia tra due valori predefiniti a seconda che l'ingresso superi una soglia superiore o scenda sotto una soglia inferiore.

Alcuni ingressi nelle FPGA sono dedicati esclusivamente ai segnali di clock, che costituiscono il riferimento per l'evoluzione temporale dell'intera rete sincrona: il clock deve essere distribuito a tutti i dispositivi in modo sincronizzato, con bassa latenza e basso skew. La rete di distribuzione adotta tipicamente una struttura ad albero bilanciato, che garantisce una propagazione uniforme del segnale, riducendo al minimo lo *skew*, definito come la differenza temporale tra i fronti del clock ricevuti dai vari dispositivi. Sebbene le logiche asincrone consumino meno potenza rispetto a quelle sincrone, le FPGA sincrone permettono di gestire complessità circuitali difficilmente raggiungibili con logiche asincrone.

7 Microprocessori e Microcontrollori

7.1 Componenti Generali di un Microcontrollore

Studiando i microcontrollori, possiamo partire da uno schema ad alto livello della sua architettura generale tipica, che comprende CPU, memorie e sistemi di I/O. La **CPU Core** è il cuore del microcontrollore: esegue le istruzioni, effettua i calcoli aritmetico/logici e gestisce il flusso dei dati, con un set di istruzioni spesso ottimizzato per gestire direttamente l'I/O. La **ROM** (o NVRAM) è la memoria non volatile on-chip, che contiene il codice del programma e il sistema operativo, i quali devono rimanere salvati anche quando si spegne il dispositivo. La **RAM**, memoria volatile on-chip, contiene lo stack e i dati temporanei su cui il programma sta lavorando in quel momento, cancellandosi allo spegnimento. Sebbene l'obiettivo sia l'integrazione, i sistemi complessi possono estendere la memoria tramite bus esterni, ma l'accesso off-chip introduce latenza e consuma più energia, per cui si preferisce massimizzare l'uso delle risorse interne.

Il sistema di I/O comprende i pin di **Input/Output**, utilizzati per la comunicazione con l'esterno, con direzione programmabile e capaci di leggere e scrivere valori alti o bassi; dato il numero limitato di pin fisici, spesso si utilizza il **Pin Multiplexing**, dove uno stesso pin è configurato via software per svolgere funzioni diverse. **Timer** e **Counter** sono registri interni configurati come contatori: il Counter viene incrementato da un segnale esterno, mentre il Timer è pilotato dal clock del sistema e, quando raggiunge il massimo valore, genera un interrupt, azzerandosi tramite l'auto-reload.

La **PWM** (Pulse Width Modulation) è una tecnica di modulazione digitale che genera una tensione media variabile tramite impulsi rettangolari di durata (*duty cycle*) variabile, facendo così sembrare il segnale digitale simile a uno analogico: ad esempio, con un'alimentazione a 5V, un duty cycle del 10% produce una tensione media percepita di 0,5V, uno del 50% ne produce 2,5V, uno del 90% ne produce 4,5V. I **Capture Inputs** sono contatori assegnati a ogni pin con il compito di contare gli eventi esterni in ingresso senza dover fare polling continuo, generando tipicamente un interrupt quando il contatore raggiunge un valore prestabilito, permettendo al sistema di reagire solo quando è necessario (ad esempio buffer pieno).

I convertitori **A/D** e **D/A** traducono, via software, i segnali analogici dell'esterno in segnali digitali leggibili dal microcontrollore, con un'accuratezza di conversione nel range di 8-12-16 bit (e viceversa per i D/A, con una precisione di 1-2 word del processore). La **UART** (Universal Asynchronous Receiver-Transmitter) è una periferica programmabile per la comunicazione seriale digitale a bassa velocità in banda base, oggi integrata direttamente come periferica on-chip, che supporta principalmente modalità asincrona utilizzando standard come l'RS232.

7.2 Protocolli di Comunicazione: SPI e I2C

Lo **SPI** (Serial Peripheral Interface) è un protocollo sincrono, quindi abbiamo un clock che dice quando leggere e scrivere i dati, rendendo la comunicazione molto più veloce e affidabile rispetto a UART o I2C. La configurazione base prevede un **Master** (che controlla il timing) e uno o più **Slave**, con quattro linee principali:

- **SCLK** (Serial Clock): generata dal master per sincronizzare la trasmissione. Determina la velocità della trasmissione: non essendoci handshaking, lo slave deve semplicemente stare al passo.
- **MOSI** (Master Output, Slave Input): linea dati in uscita dal master e in ingresso allo slave.
- **MISO** (Master Input, Slave Output): linea dati in uscita dallo slave e in ingresso al master.

- **SS** (Slave Select): linea dedicata per ogni slave, che il master porta a livello basso per selezionare lo slave desiderato.

Il protocollo **I2C** (Inter-Integrated Circuit), sviluppato da Philips (ora NXP), è un bus seriale sincrono a due fili progettato per collegare periferiche a bassa velocità come sensori, RTC ed EEPROM, minimizzando il numero di pin. A differenza dello SPI, l'I2C utilizza un'architettura **Open-Drain** con resistenze di pull-up esterne: i dispositivi possono solo forzare le linee a livello basso, mentre il livello alto è ripristinato passivamente dalle resistenze, evitando cortocircuiti in configurazione multi-master e permettendo di collegare dispositivi con tensioni diverse. La selezione dello slave avviene via software: il master segnala l'inizio tirando giù **SDA** (Serial Data) mentre **SCL** (Serial Clock) è alto, poi invia 7 bit di indirizzo più 1 bit di lettura/scrittura, e dopo ogni byte il ricevitore deve tirare giù **SDA** per confermare la ricezione (*acknowledge*).

7.3 Il Watchdog Timer

Il **Watchdog Timer** (WDT) è un timer hardware autonomo progettato per rilevare anomalie software (loop infiniti, deadlock, crash) e hardware (glitch di alimentazione). Per garantire la massima sicurezza, il WDT è solitamente alimentato da una sorgente di clock interna indipendente separata dal clock principale del sistema, così da poter resettare il sistema anche in caso di guasto totale. Il WDT conta alla rovescia partendo da un valore preimpostato: il software, durante il suo normale ciclo di funzionamento, deve periodicamente scrivere un valore specifico in un registro del WDT per riportare il contatore all'inizio. Se ciò accade, il sistema continua normalmente; se non succede, scatta l'azione di sicurezza del WDT, consistente in un interrupt non-mascherabile che mette la macchina in sicurezza.

7.4 Organizzazione della Memoria

Ogni microprocessore o microcontrollore possiede una mappa predefinita del proprio spazio di memoria, sia interna che esterna. Questo comporta una **mappatura delle periferiche**, con le periferiche assegnate a specifici indirizzi nello spazio di memoria e correttamente abilitate quando il processore accede al loro indirizzo (processo noto come *decodifica degli indirizzi*), e un **partizionamento della memoria**, con lo spazio suddiviso in sezioni predefinite: memoria di sistema (area protetta riservata al firmware), memoria utente (area libera per l'esecuzione dei programmi) e stack (area organizzata a "pila", essenziale per gestire chiamate a funzione, salti e ritorni).

Il microprocessore comunica con la memoria esterna tramite bus di indirizzi e dati: questo diventa costoso, poiché un processore a 16 bit con 16 bit di indirizzi avrebbe bisogno di 32 pin solo per il bus. Per ridurre il numero di pin del package si utilizza un **bus multiplexato**, dove gli stessi 16 pin portano prima l'indirizzo e poi il dato. Nella fase Q1 del primo ciclo, la CPU pone l'indirizzo sulle linee del bus e attiva il segnale **ALE** (Address Latch Enable), che va alto dicendo che sul bus c'è un indirizzo; a questo punto i latch diventano trasparenti, lasciando passare il segnale, e quando **ALE** va basso i latch si chiudono, memorizzando l'ultimo valore visto e inviandolo alla memoria. Un blocco decoder legge alcuni bit dell'indirizzo e decide quale chip di memoria attivare tramite il segnale **CE** (Chip Enable). Ora che l'indirizzo è stato salvato, le linee cambiano funzione diventando data bus. I segnali di controllo principali sono **OE** (Output Enable), che abilita la lettura, **WR** (Write), che abilita la scrittura, e **CE**.

La decodifica degli indirizzi consente al microcontrollore di selezionare specifiche sezioni di memoria o periferiche: con un bus di indirizzi a 16 bit è possibile indirizzare $2^{16} = 65536$ combinazioni, ovvero 64KB. Se si utilizza un chip di memoria da 8KB, sono necessari 13 bit di indirizzo per coprire l'intero spazio ($2^{13} = 8192$ word), e i 3 bit rimanenti vengono usati per selezionare quale tra 8 chip attivare, tramite un segnale di Chip Enable generato da una

funzione combinatoria degli indirizzi. È inoltre possibile configurare il microcontrollore per ignorare certe aree di memoria, escludendo dalla mappa le letture/scritture verso quelle zone.

7.5 Modalità di Interfaccia Periferica

Nei sistemi embedded, la comunicazione tra processore e periferiche può avvenire tramite tre modalità principali, ciascuna con vantaggi e svantaggi. Il **Polling** è una tecnica ciclica in cui il processore verifica continuamente lo stato di ciascuna periferica: è semplice e non richiede hardware complesso, ma è inefficiente perché la CPU rimane costantemente impegnata nel controllo delle periferiche, riducendo le risorse disponibili per altre operazioni, risultando incompatibile con il multitasking e con un costo in tempo e consumi che cresce con il numero di periferiche gestite.

L'**Interrupt** è un meccanismo hardware che consente alla CPU di reagire ad eventi asincroni, sospendendo il programma corrente, eseguendo la routine di interrupt e riprendendo poi l'esecuzione. Esistono interrupt **mascherabili**, che possono essere disattivati temporaneamente durante l'esecuzione di una routine critica, e interrupt **non mascherabili** (NMI) per processi prioritari che non possono essere disattivati, come surriscaldamento o caduta di alimentazione; esiste inoltre il concetto di **preemption** o *nesting*, per cui un interrupt ad alta priorità può interrompere una ISR a bassa priorità già in esecuzione, creando una struttura a “pila” di interruzioni annidate. L'interrupt ottimizza l'uso della CPU, che non è costretta a controllare ciclicamente le periferiche, ma richiede un sistema di gestione degli interrupt e, se la loro frequenza è eccessiva, rallenta il sistema.

Il **DMA** (Direct Memory Access) è un modulo hardware specializzato incaricato di gestire il trasferimento dati ad alta velocità tra memoria e periferiche senza l'intervento attivo della CPU: la CPU avvia il trasferimento configurando il controller DMA, il quale gestisce autonomamente il trasferimento di blocchi di dati, mentre la CPU interviene solo per monitorare il processo. È molto efficiente, riducendo il carico di interrupt sulla CPU e permettendo il trasferimento rapido di grandi quantità di dati, ma richiede hardware dedicato ed è più complesso da implementare rispetto alle altre modalità.

8 Digital Signal Processors (DSP) e SoC

8.1 Motivazioni e Peculiarità

I **DSP** (Digital Signal Processors) sono microprocessori progettati specificamente per l'elaborazione numerica dei segnali, nati dalla necessità di tradurre le operazioni sui segnali da sistemi analogici a digitali, sfruttando la maggiore flessibilità, prevedibilità e stabilità offerte dal software digitale rispetto agli equivalenti sistemi hardware analogici. I DSP consentono di modificare parametri e algoritmi in modo dinamico senza la necessità di cambiare l'hardware, rendendo queste architetture ideali per una vasta gamma di applicazioni: telecomunicazioni, controllo di processo, radar, automotive e gestione di controllori.

Il successo dei DSP si basa su quattro pilastri fondamentali. La **prevedibilità** deriva dall'elaborazione matematica pura: a parità di input, l'output è garantito, senza le tolleranze dei componenti fisici. La **stabilità** significa che i sistemi digitali non soffrono di deriva termica né di invecchiamento, garantendo prestazioni costanti nel tempo. La **flessibilità** permette di modificare il comportamento del sistema tramite un semplice aggiornamento software anziché la sostituzione fisica di hardware. La **compattezza**, infine, grazie alla tecnologia VLSI, permette di integrare funzioni complesse in un singolo chip minuscolo.

Le peculiarità hardware dei DSP includono hardware dedicato per operazioni **MAC** (Multiply and Accumulate), essenziale per operazioni matematiche ripetitive ad alta velocità come la convoluzione; accessi multipli alla memoria, tramite l'architettura Harvard, che garantisce

un flusso continuo senza attese; modalità di indirizzamento dedicate, per facilitare il trattamento di flussi di dati continui; strutture di controllo dedicate, dove l'hardware gestisce i cicli senza sprecare cicli di clock per decrementare contatori e gestire i salti; periferiche dedicate on-chip come ADC e DAC per ridurre la latenza; e un'alta frequenza di funzionamento, rendendo i DSP adatti ad applicazioni ad alta intensità computazionale come radar e telecomunicazioni.

8.2 DSP su System on Chip

Invece di avere chip separati sulla scheda madre, con i **DSP SoC** si integra tutto su un unico pezzo di silicio, combinando un DSP Core (dedicato all'elaborazione matematica dei segnali) con un microcontrollore (che gestisce la logica di controllo). Il chip include direttamente convertitori A/D e D/A per interfacciarsi con l'esterno, memoria RAM e ROM condivisa o dedicata, custom logic per blocchi digitali o analogici personalizzati, e porte seriali per l'I/O. I tre vantaggi principali di questa integrazione sono: **efficienza**, grazie all'uso di core con architettura RISC per il microcontrollore, che garantisce un'esecuzione rapidissima delle istruzioni elementari di controllo; **flessibilità**, grazie al supporto di istruzioni complesse di tipo CISC che permettono di gestire algoritmi avanzati senza codice eccessivamente lungo; e **compattezza ed economicità**, dato che mettere tutto su un solo chip riduce drasticamente costi, dimensioni e consumo energetico.

8.3 Aritmetica DSP e Unità MAC

L'aritmetica dei processori DSP si divide in due grandi famiglie. Nella **Fixed Point** (virgola fissa), il processore tratta i numeri come se fossero interi puri, con la posizione della virgola gestita dal programmatore, tipicamente a 16, 20 o 24 bit: è la scelta per applicazioni come la telefonia, dove la dinamica del segnale non è estrema ma il risparmio energetico è cruciale. Nella **Floating Point** (virgola mobile), il processore gestisce automaticamente mantissa ed esponente, solitamente a 32 bit secondo lo standard IEEE 754, offrendo una gamma dinamica enormemente superiore, ideale per audio hi-fi, grafica 3D e radar.

Il **MAC** (Multiply and Accumulate) è una componente hardware essenziale nei DSP, progettata per eseguire operazioni di moltiplicazione e somma in un singolo ciclo di clock. Ad esempio, in un'operazione con numeri complessi, il MAC riceve due bus con parte reale e immaginaria, li moltiplica creando i prodotti parziali, esegue lo stadio di somma per calcolare parte reale e immaginaria del prodotto complesso, e infine due accumulatori separati con feedback sommano i risultati parziali, completando tutto in un unico ciclo di clock.

8.4 Architetture di Memoria: Von Neumann e Harvard

L'architettura di **Von Neumann** è caratterizzata dall'utilizzo di una singola memoria condivisa per dati e istruzioni, connessa al processore tramite un unico bus, utilizzato alternativamente per accedere alle istruzioni e ai dati. È semplice da progettare, riducendo complessità e costo, ma l'accesso condiviso crea un collo di bottiglia: il processore non può leggere un'istruzione e caricare un dato nello stesso istante, dovendo fare il fetch dell'istruzione al ciclo corrente e leggere il dato al ciclo successivo, il che rende questa architettura inefficiente per i DSP, dove l'elaborazione in tempo reale richiede una velocità di accesso superiore.

L'architettura **Harvard** separa fisicamente le memorie per dati e istruzioni, ognuna con il proprio bus dedicato, eliminando il collo di bottiglia dell'architettura di Von Neumann e permettendo accessi paralleli: il processore può leggere istruzioni e dati contemporaneamente, aumentando la velocità di esecuzione e risultando adatta per applicazioni real-time, a costo di un hardware maggiore per la duplicazione di bus e memorie. Una variante avanzata è l'architettura **Harvard con Triple Data Bus / Dual-port**, che prevede l'uso di memorie dual-port capaci di accessi simultanei alla stessa memoria: si utilizza un bus per le istruzioni

e due bus per i dati, permettendo la lettura simultanea di due operandi e la scrittura del risultato.

8.5 Modalità di Indirizzamento DSP

I DSP presentano modalità di indirizzamento non comuni nei microprocessori convenzionali, dovute ai requisiti software di alto livello richiesti dagli algoritmi di elaborazione del segnale digitale. L'**indirizzamento immediato** utilizza una costante specificata direttamente all'interno dell'istruzione. L'**indirizzamento indiretto** usa un registro che contiene l'indirizzo di memoria desiderato. L'**indirizzamento pre-post incremento** è progettato per facilitare l'accesso a sequenze di dati, automatizzando il passaggio alla variabile successiva o precedente senza doverlo specificare esplicitamente in ogni istruzione. L'**indirizzamento circolare** è particolarmente utile per la gestione di buffer circolari, comunemente utilizzati negli algoritmi di elaborazione del segnale come le code FIFO: un registro puntatore mantiene il puntatore all'interno di un intervallo definito, e quando il buffer raggiunge il suo limite il puntatore ritorna automaticamente all'inizio. L'**indirizzamento con inversione di bit**, infine, è progettato per ottimizzare l'implementazione della Trasformata Rapida di Fourier (FFT), riorganizzando i dati in base all'inversione dei bit del loro indirizzo.

Il set di istruzioni dei DSP include istruzioni non standard progettate per ottimizzare l'elaborazione di segnali e algoritmi complessi: il MAC, i **Block Floating Point**, che permettono di gestire blocchi di memoria con maggiore precisione assegnando un unico esponente comune a un intero blocco di dati; gli **Hardware Loops**, che consentono l'esecuzione di cicli direttamente in hardware senza necessità di istruzioni software per l'incremento del contatore, accelerando significativamente le operazioni iterative; i **Nested Hardware Loops**, che implementano cicli annidati direttamente in hardware; e il **Data Block Movement**, che facilita il trasferimento rapido di blocchi di dati.

9 Architettura MIPS

9.1 Il Design Quantitativo del Microprocessore

Il **Micro Processor Quantitative Design** è un approccio sistematico all'analisi, progettazione e ottimizzazione dei microprocessori basato su principi quantitativi, che si concentra sulla misurazione e valutazione delle prestazioni, dell'efficienza e dei compromessi di progettazione, utilizzando metriche come il numero di cicli di clock per istruzione, la latenza e il throughput. L'obiettivo è massimizzare l'efficienza del processore attraverso l'analisi dettagliata delle istruzioni, della pipeline e delle unità funzionali, bilanciando prestazioni con costi di implementazione in termini di risorse hardware e consumo energetico.

Legge/Teorema 9.1 (Speedup). *Lo speedup quantifica il miglioramento delle prestazioni di un sistema grazie a una nuova soluzione hardware o architetturale, ed è definito come il rapporto tra il tempo di esecuzione senza la nuova soluzione ($t_{without}$) e il tempo di esecuzione con la nuova soluzione (t_{with}):*

$$S = \frac{t_{without}}{t_{with}}$$

Affinché la nuova soluzione sia vantaggiosa deve risultare $t_{with} < t_{without}$, ossia $S > 1$. Le scelte architetturelle devono essere qualificate in base al loro impatto benefico e in base al rapporto tra costo e prestazioni, testando l'architettura sull'esecuzione di software benchmark standard, chiamati high level requirements.

Legge/Teorema 9.2 (Legge di Amdahl). *La legge di Amdahl definisce il limite teorico massimo di miglioramento di un sistema, anche quando viene ottimizzata solo una parte del pro-*

cesso: il miglioramento totale è vincolato dalla parte di programma che non può usare quella componente. L'equazione è

$$S = \frac{1}{(1 - f_{enh}) + \frac{f_{enh}}{S_{enh}}}$$

dove f_{enh} è la frazione di codice ottimizzata e S_{enh} è lo speed-up della parte ottimizzata. Nel caso in cui $f_{enh} = 1$, cioè la frazione ottimizzata è massima, il termine $(1 - f_{enh})$ si annulla e rimane $S = S_{enh}$.

Consideriamo un esempio pratico: una CPU spende il 40% del suo tempo in elaborazione dei numeri e il 60% nell'aspettare i dati di I/O. Quanto migliora la situazione adottando una CPU dieci volte più veloce sulla parte di elaborazione? Con $f_{enh} = 0.4$ e $S_{enh} = 10.0$:

$$S = \frac{1}{(1 - 0.4) + \frac{0.4}{10}} \approx 1.56$$

Un secondo esempio, ancora più istruttivo, riguarda una CPU che spende il 20% del tempo eseguendo radici quadrate in virgola mobile ($f_{enh} = 0.2$) e il 50% del tempo in altre istruzioni a virgola mobile ($f_{enh} = 0.5$). Migliorando le radici quadrate con un acceleratore hardware di fattore 10 ($S_{enh} = 10$): $S = 1/[(1 - 0.2) + (0.2/10)] \approx 1.22$. Migliorando invece tutte le altre istruzioni FP di un fattore 1.6 ($S_{enh} = 1.6$): $S = 1/[(1 - 0.5) + (0.5/1.6)] \approx 1.23$. In questo caso migliorare il blocco più grande è leggermente più vantaggioso, mostrando come i limiti del miglioramento dipendano fortemente dalla frazione non ottimizzata.

Legge/Teorema 9.3 (Equazione delle Prestazioni, o Iron Law). *Il tempo di CPU è l'unica metrica affidabile per valutare le prestazioni, ed è calcolato come il prodotto di tre fattori:*

$$CPU_time = IC \times CPI \times T_{cycle}$$

dove IC è il numero di istruzioni (il volume di lavoro software), CPI è il numero medio di cicli di clock per istruzione (misura l'efficienza architetturale, con un CPI ideale pari a 1 ma reale maggiore di 1 a causa di stalli di memoria o conflitti), e T_{cycle} è la durata del ciclo di clock. Ogni fattore dell'equazione è influenzato da specifici aspetti del design: IC dipende dall'ISA e dalla capacità del compilatore di tradurre il codice in modo conciso; CPI dipende dall'organizzazione interna (pipeline, cache) e dall'ISA; il tempo di ciclo dipende dalla tecnologia hardware e dall'organizzazione.

9.2 Evoluzione delle Architetture

Negli anni '80, le architetture predominanti erano quelle basate sull'accumulatore, dovuto al fatto che le tecnologie integrate erano agli albori e non permettevano soluzioni più complesse. Con il progredire della tecnologia emersero architetture più sofisticate note come **CISC** (Complex Instruction Set Computer), progettate per eseguire operazioni complesse in un minor numero di istruzioni. Tuttavia, agli inizi degli anni '90 si dimostrò che mantenere l'hardware semplice e veloce, approccio noto come **RISC** (Reduced Instruction Set Computer), garantiva migliori prestazioni: fu in questo contesto che iniziarono a diffondersi le architetture register-to-register, che sfruttavano appieno la località dei dati introducendo l'uso delle memorie cache, riducendo drasticamente gli accessi alla memoria centrale.

Le architetture di microprocessore si dividono in **stack** (i dati vengono memorizzati in una pila, con operazioni push/pop sulla cima), **accumulator** (un operando in un registro dedicato, l'altro in memoria), **register-memory** (gli operandi possono essere registri o locazioni di memoria), e **register-register** (entrambi gli operandi risiedono nei registri, riducendo l'accesso alla memoria). Le architetture RISC come MIPS favoriscono soluzioni register-register per la loro semplicità e velocità.

9.3 Endianness e Allineamento

Si può definire l'ordine con cui i byte che compongono una “word” vengono disposti nella memoria lineare. Nel **Big-Endian**, il byte più significativo va all'indirizzo di memoria più basso; nel **Little-Endian**, è il byte meno significativo a occupare l'indirizzo più basso.

La velocità di accesso alla memoria dipende anche dall'allineamento: un dato è allineato se il suo indirizzo di memoria è un multiplo della sua dimensione. Un byte (1 byte) è sempre allineato; una half word (2 byte) è allineata solo se l'indirizzo è pari; una word (4 byte) è allineata solo se l'indirizzo è divisibile per 4; una double word (8 byte) è allineata solo se l'indirizzo è divisibile per 8. Se un dato da 4 byte viene collocato a un indirizzo non multiplo di 4 (*misaligned*), il processore deve fare uno sforzo extra per recuperarlo.

9.4 Modalità di Indirizzamento MIPS

Nel MIPS distinguiamo diverse modalità di indirizzamento. L'**indirizzamento a registro** è usato quando il valore su cui lavorare è già stato caricato in un registro: è la modalità più veloce perché non richiede accesso alla memoria (**Add R4, R3**). L'**indirizzamento immediato** include l'operando come costante numerica direttamente nell'istruzione (**Add R4, #3**). L'**indirizzamento a spiazzamento** (displacement) calcola l'indirizzo sommando il contenuto di un registro con un valore costante di offset, fondamentale per accedere a variabili locali nello stack o a strutture dati (**Add R4, 100(R1)**). L'**indirizzamento indiretto a registro** usa un registro che contiene non il dato ma l'indirizzo di memoria dove si trova il dato (**Add R4, (R1)**). L'**indirizzamento indicizzato** calcola l'indirizzo finale come somma del contenuto di due registri, uno che funge da base e l'altro da indice, utile per l'accesso agli array (**Add R3, (R1+R2)**). L'**indirizzamento diretto o assoluto** contiene direttamente l'indirizzo di memoria completo (**Add R1, (1001)**). L'**indirizzamento indiretto a memoria** usa un registro che punta a una cella di memoria contenente l'indirizzo del dato finale, un concetto di puntatore a puntatore. L'**autoincremento** e l'**autodecremento** sono simili al register indirect, ma dopo l'accesso il registro viene automaticamente incrementato o decrementato della dimensione dell'elemento, utile nei loop per scorrere gli array. L'**indirizzamento scalato**, infine, calcola l'indirizzo come $\text{Base} + \text{Offset} + (\text{Indice} \times \text{Scala})$.

L'approccio quantitativo misura la frequenza d'uso dei vari modi di indirizzamento: i dati evidenziano che Displacement (accesso a variabili locali/strutture) e Immediate (uso di costanti) rappresentano oltre l'80% degli accessi. Di conseguenza, le architetture RISC moderne sono ottimizzate per eseguire queste operazioni nel minor tempo possibile, mentre modi complessi come Memory Indirect o Scaled, avendo frequenze di utilizzo trascurabili, vengono eliminati e lasciati emulare al compilatore tramite sequenze di istruzioni più semplici, senza impattare le prestazioni globali. Studi statistici mostrano inoltre che nella maggior parte delle funzioni il 96% del carico di lavoro totale è svolto da sole 10 istruzioni, con le operazioni più frequenti rappresentate dal trasferimento dati (Load 22%, Store 12%) e dal controllo del flusso (Conditional Branch 20%, Compare 16%), mentre le operazioni aritmetiche complesse sono rare.

9.5 L'Interfaccia Software/Hardware e le Istruzioni MIPS

L'interfaccia tra software e hardware, definita come **Instruction Set Architecture (ISA)**, specifica come il software interagisce con i componenti hardware. Un'architettura come MIPS adotta una struttura fissa delle istruzioni (32 bit) per semplificare la progettazione, evitando la complessità di istruzioni di lunghezza variabile.

Il set di istruzioni MIPS suddivide le operazioni in tre formati principali. Il **Tipo R** (Register) è usato per operazioni puramente aritmetico-logiche che lavorano sui dati presenti nei registri: la sua struttura divide i 32 bit in opcode (0 per R-type), registri sorgente (**rs, rt**),

registro destinazione (**rd**), **shamt** (shift amount) e **funct** (che specifica l'operazione esatta). Il **Tipo I** (Immediate) è usato quando uno degli operandi è un numero costante o per calcolare indirizzi di memoria: sostituisce il terzo registro con un campo a 16 bit per il valore. Il **Tipo J** (Jump), infine, è dedicato ai salti incondizionati a indirizzi lontani, dedicando la maggior parte dei bit (26) all'indirizzo di destinazione.

Legge/Teorema 9.4 (Böhm-Jacopini). *Qualsiasi algoritmo può essere implementato combinando solo tre strutture fondamentali: la **sequenza** (esecuzione atomica delle istruzioni in ordine lineare), la **selezione** (blocchi di codice alternativi basati su una condizione booleana, struttura if-then-else) e l'**iterazione** (ripetizione di un blocco di istruzioni finché una condizione rimane vera).*

Le istruzioni MIPS si dividono in cinque classi funzionali. Le istruzioni **Arithmetic** gestiscono i calcoli interni: somma e sottrazione (**add**, **sub**) e le versioni con costanti immediate (**addi**). Le istruzioni **Logical** operano sui singoli bit (**and**, **or**, **nor**): si usa **and/andi** per isolare bit specifici e **or/ori** per settare bit a 1; è utile notare che MIPS non ha un NOT diretto, ma si usa una **nor** con il registro zero ($A \text{ NOR } 0 = \text{NOT } A$); vi sono anche shift logici (**sll**, **srl**) utili per manipolazioni rapide e moltiplicazioni per potenze di 2. Le istruzioni **Data Transfer**, essendo MIPS un'architettura Load/Store, sono le uniche che accedono alla RAM: possiamo caricare/salvare intere word (**lw/sw**), half word (**lh/sh**) o singoli byte (**lb/sb**). Le istruzioni **Conditional Branch** (Tipo I) sono usate per i salti con confronto, relativi al Program Counter (**beq**, **bne**). Le istruzioni **Unconditional Jump** (Tipo J), infine, saltano senza condizione: **j** (salto diretto), **jal** (usato per le chiamate a funzione), e **jr** (salta all'indirizzo contenuto in un registro, usato per il ritorno da una funzione o per implementare uno switch-case).

9.6 L'Architettura Single-Cycle

L'architettura a ciclo singolo MIPS esegue ogni istruzione in un singolo ciclo di clock, la cui durata è determinata dal percorso critico, ovvero dall'istruzione con la latenza più alta: $T_{clk} \leq T_{max_istruzione}$. Nonostante l'architettura sia sincrona con il tempo di clock, le azioni eseguite al suo interno sono asincrone. Questo modello garantisce semplicità ma penalizza le istruzioni con latenza inferiore, perché le vincola alla durata del caso peggiore.

I componenti base includono la **Instruction Memory**, che contiene il codice macchina del programma e fornisce l'istruzione a 32 bit memorizzata all'indirizzo ricevuto in input; il **Program Counter** (PC), un registro a 32 bit che contiene l'indirizzo dell'istruzione corrente e viene aggiornato alla fine di ogni ciclo; l'**Adder**, un circuito combinatorio aritmetico; i **Registers** (Register File), il banco dei 32 registri generali, progettato per permettere la lettura simultanea di due registri e la scrittura di uno, controllata dal segnale **RegWrite**; la **ALU** (Arithmetic Logic Unit), l'unità esecutiva che riceve due operandi e produce sia il risultato dell'operazione sia un flag **Zero**, che vale 1 se il risultato è zero (utilizzato per i salti condizionali **beq**); la **Data Memory Unit**, indirizzata dal risultato della ALU, che supporta lettura (**MemRead**) e scrittura (**MemWrite**); e la **Sign-extension Unit**, che converte un valore immediato a 16 bit in un valore a 32 bit, preservando il valore numerico in complemento a due tramite la replica del bit di segno.

Il datapath si articola in quattro sezioni. La **Fetch Section**, composta da PC, Instruction Memory e un Adder, si occupa di prelevare l'istruzione ad ogni ciclo di clock e di determinare l'indirizzo della prossima istruzione: mentre l'Instruction Memory restituisce l'istruzione corrispondente al PC, in parallelo un adder calcola $PC+4$. Tutti i blocchi, ad eccezione del PC, sono combinatori. L'**Arithmetic Section**, composta da Registers e ALU, esegue le operazioni aritmetico-logiche richieste dalle istruzioni: il register file per la lettura riceve gli indici dei registri e fornisce i dati contenuti, mentre per la scrittura riceve l'indice del registro di

destinazione e il dato da salvare, attivata solo se **RegWrite** è alto. La **Data Memory Access Section** aggiunge Sign Extend e Data Memory: il Sign-extension Unit converte l'offset a 32 bit, la ALU calcola l'indirizzo effettivo sommando base e offset, e la Data Memory esegue lettura o scrittura in base ai segnali **MemRead/MemWrite**. La **Conditional Branch Section**, infine, aggiunge una seconda ALU, un blocco di Sign Extend e uno Shift Left di 2: dai registri vengono letti i due valori da confrontare, la seconda ALU esegue una sottrazione (interessa solo il flag Zero), il Sign-extension Unit e lo Shift Left 2 preparano l'offset (moltiplicato per 4, poiché 1 istruzione = 4 byte), e infine un adder calcola l'indirizzo di destinazione con la formula $\text{Branch Target} = (PC + 4) + (\text{Offset} \times 4)$.

L'integrazione delle quattro sezioni in un unico datapath avviene attraverso il principio della condivisione delle risorse: invece di avere tre ALU separate per ogni tipo di operazione, si utilizza una singola ALU principale per calcoli aritmetici, calcolo degli indirizzi e confronti per i salti. Per gestire questo traffico condiviso si introducono i multiplexer, dispositivi di selezione che, guidati dai segnali di controllo, decidono istante per istante quale dato inviare alla ALU o scrivere nei registri: **RegDst** seleziona il campo che indica il registro di destinazione (**rd** o **rt**), **ALUSrc** seleziona il secondo operando della ALU (il valore letto dai registri o l'offset esteso), **MemtoReg** seleziona la fonte del dato da scrivere nel registro finale (l'uscita della ALU o il dato letto dalla memoria), e **PCSrc** seleziona il nuovo valore del PC tra sequenziale, branch e jump. Una Control Unit genera tutti i segnali di controllo a partire dall'opcode dell'istruzione, mentre una ALU Control decodifica il campo **funct** per le istruzioni di Tipo R.

9.7 L'Architettura Multi-Cycle

L'architettura **Single-Cycle**, per quanto semplice, è inefficiente. L'architettura **Multi-Cycle**, invece, spezza l'esecuzione di un'istruzione in una serie di passi elementari e sequenziali, con un ciclo di clock molto più breve, calibrato sulla durata dei singoli step. Poiché l'esecuzione si spalma nel tempo, il sistema non è più puramente combinatorio ma diventa una macchina a stati finiti: il processore deve “ricordarsi” a che punto è arrivato dell'esecuzione. Per far ciò, vengono introdotti registri temporanei posti tra le unità funzionali, al fine di memorizzare i risultati parziali affinché siano disponibili come input per il ciclo successivo.

Nel datapath multi-ciclo completo si opera un'unificazione delle risorse: non esistono più Instruction Memory e Data Memory separate, ma un unico blocco Memory per istruzioni e dati, il cui accesso è gestito da un multiplexer **IorD** che seleziona l'indirizzo proveniente dal PC (Fetch) o dall'**ALUOut** (Load/Store). Il numero di ALU si riduce a una sola che esegue tutte le operazioni: incremento del PC, calcolo degli indirizzi, confronti per i branch e operazioni aritmetiche, con i multiplexer **ALUSrcA** e **ALUSrcB** che selezionano gli ingressi corretti ciclo per ciclo. Per “parcheggiare” i dati intermedi si aggiungono registri temporanei non architetturali: l'**Instruction Register** (IR) mantiene l'istruzione corrente per tutta la durata dell'esecuzione; il **Memory Data Register** (MDR) salva il dato letto dalla memoria prima di scriverlo nel Register File; A e B sono buffer per gli operandi letti dal Register File; e **ALUOut** è un buffer per il risultato della ALU. L'aggiornamento del PC non è più automatico ad ogni ciclo, ma controllato selettivamente tramite i segnali **PCWrite** (forza la scrittura del PC, usato nel Fetch e per il Jump), **PCWriteCond** (abilita la scrittura solo se la condizione di salto è vera, **beq**) e **PCSource** (un multiplexer a 3 vie che sceglie il nuovo valore del PC tra risultato ALU, registro **ALUOut** o indirizzo di Jump).

Il flusso di esecuzione si articola in cinque fasi:

- **Fase 1 – Instruction Fetch** (comune a tutti): si preleva l'istruzione dalla memoria e la si salva nel registro IR ($IR \leftarrow Memory[PC]$). Contemporaneamente, si usa la ALU per calcolare $PC + 4$ e aggiornare il PC ($PC \leftarrow PC + 4$): ciò è possibile perché memoria e ALU sono risorse separate in quel momento.

- **Fase 2 – Decode & Register Fetch** (comune a tutti): mentre la Control Unit decodifica l'opcode, il datapath legge i registri **rs** e **rt** salvandoli nei buffer temporanei A e B ($A \leftarrow \text{Reg}[\text{IR}[25 : 21]]$; $B \leftarrow \text{Reg}[\text{IR}[20 : 16]]$), ed esegue il pre-calcolo del branch target con l'ALU ($\text{ALUOut} \leftarrow PC + (\text{SignExt}(\text{Imm}) \ll 2)$), calcolo che viene semplicemente ignorato se l'istruzione non è un branch.
- **Fase 3 – Execution** (specifica per opcode): per le istruzioni *Memory Reference* (**lw/sw**) la ALU calcola l'indirizzo fisico ($\text{ALUOut} \leftarrow A + \text{sign_extend}(\text{IR}[15 : 0])$); per le istruzioni **R-Type** la ALU esegue l'operazione ($A \text{ op } B$) salvando in **ALUOut**; per il **Branch** la ALU esegue la sottrazione $A - B$, e se il risultato è zero il PC viene aggiornato con il valore calcolato nella fase precedente ($\text{if}(A == B) \text{ PC} \leftarrow \text{ALUOut}$); per il **Jump** il PC viene sovrascritto concatenando i bit del PC corrente con l'indirizzo nell'istruzione ($\text{PC} \leftarrow \text{PC}[31 : 28] \& (\text{IR}[25 : 0] \& "00")$).
- **Fase 4 – Memory Access**: l'istruzione **lw** legge il dato dalla memoria all'indirizzo calcolato (in **ALUOut**) e lo salva nell'MDR ($\text{MDR} \leftarrow \text{Memory}[\text{ALUOut}]$), necessario perché la lettura della memoria occupa l'intero ciclo; l'istruzione **sw** scrive il valore del registro B in memoria ($\text{Memory}[\text{ALUOut}] = B$); le istruzioni R-Type scrivono direttamente il valore contenuto in **ALUOut** nel registro di destinazione **rd** ($\text{Reg}[\text{IR}[15 : 11]] \leftarrow \text{ALUOut}$).
- **Fase 5 – Write Back** (solo Load): sposta il dato “parcheggiato” nel Memory Data Register verso il registro di destinazione finale **rt** nel Register File.

10 Gerarchia di Memoria e Cache

10.1 Il Principio di Località

La memoria cache è un componente essenziale dell'architettura dei sistemi di calcolo, progettata per migliorare le prestazioni della CPU riducendo i tempi di accesso alla memoria. L'idea di base è semplice: tenere a portata di mano i dati usati di frequente, sfruttando le metriche di **località temporale**, per cui i dati recentemente utilizzati hanno un'alta probabilità di essere riutilizzati, e **località spaziale**, per cui i dati vicini a quelli recentemente utilizzati saranno probabilmente richiesti in breve tempo, motivo per cui la cache preleva interi blocchi di memoria che contengono sia il dato richiesto sia quelli adiacenti.

La **gerarchia di memoria** è una struttura a piramide progettata per bilanciare tre fattori contrastanti: velocità, capacità e costo. Poiché non esiste una tecnologia di memoria che sia contemporaneamente velocissima, enorme ed economica, si utilizzano più livelli: il **Livello 1** (L1), integrato nel core della CPU, offre il tempo di accesso minimo ma ha una capacità molto ridotta; il **Livello 2** (L2) funge da cuscinetto intermedio; e ai livelli superiori troviamo memorie più capienti ma drasticamente più lente, come la RAM e la memoria secondaria. Al variare dei livelli, aumenta la distanza dalla CPU, e con essa aumentano i tempi di accesso e la dimensione delle memorie.

Le **figure di merito** che permettono di giudicare l'efficacia di una cache sono la *capacità* (dimensione fisica della memoria), l'*Hit Rate* (percentuale di successo nel trovare i dati al livello attuale), la *Miss Rate* (pari a $1 - \text{Hit Rate}$), l'*Hit Time* (latenza minima di accesso quando il dato è presente) e la *Miss Penalty* (il costo temporale aggiuntivo richiesto per recuperare il dato dal livello inferiore).

10.2 Architetture di Cache

Nell'architettura **Direct Mapped**, ogni linea della cache può memorizzare un solo blocco di dati: ogni blocco di memoria principale è assegnato a una specifica riga tramite l'*Index*

dell'indirizzo. La struttura dell'indirizzo (32 bit) si suddivide in **Tag** (identifica il blocco di memoria), **Index** (indica quale linea della cache contiene il dato richiesto) e **Byte Offset** (seleziona il byte specifico). In un tipico esempio didattico a parola singola, l'indirizzo a 32 bit si scompone in Tag (bit 31–12, 20 bit), Index (bit 11–2, 10 bit, che selezionano una fra $2^{10} = 1024$ righe) e Byte Offset (bit 1–0, per selezionare il byte all'interno della parola a 32 bit). Il funzionamento prevede che l'Index individui la linea della cache, il Tag venga confrontato (tramite un comparatore) con quello memorizzato nella linea selezionata (se il confronto è positivo e la linea è valida si ha un *hit*, altrimenti un *miss*), e il Byte Offset selezioni (tramite multiplexer) il byte specifico all'interno del blocco. Questa architettura è veloce ed economica da implementare, poiché richiede un solo confronto di Tag per ogni accesso, ma è soggetta a conflitti se più dati necessari al programma competono per la stessa riga.

L'architettura **Multi-block** (o Multi-word Block) mantiene la logica della mappatura diretta ma espande la dimensione della riga di cache, che non contiene più una singola parola (32 bit) ma un blocco più grande (ad esempio 512 bit totali di dati per riga). L'indirizzo a 32 bit si reinterpreta quindi come: **Tag** (ad esempio 18 bit, più piccolo rispetto al caso a parola singola perché il blocco dati è più grande), **Index** (ad esempio 8 bit, che selezionano una fra $2^8 = 256$ righe disponibili), **Block Offset** (ad esempio 4 bit, che individuano quale delle $2^4 = 16$ parole da 32 bit contenute nella riga da 512 bit va inviata alla CPU tramite un multiplexer) e **Byte Offset** (2 bit, standard per architetture a 32 bit). Il vantaggio di questa architettura è di massimizzare la *località spaziale*: un solo Tag copre 512 bit di dati, quindi se si legge l'elemento `Array[0]` la cache carica automaticamente anche gli elementi da `Array[1]` a `Array[15]` nella stessa linea, rendendo immediati i successivi accessi senza interrogare nuovamente la RAM. Lo svantaggio è una **Miss Penalty** più alta, perché caricare 512 bit dalla RAM è molto più lento che caricarne 32: se avviene un miss, la CPU deve aspettare che l'intero blocco venga trasferito.

L'architettura **Set Associative** organizza la cache in S insiemi (*set*), dove ogni insieme contiene N blocchi (*way*): combina le caratteristiche della Direct Mapped e della Fully Associative. L'Index seleziona il set in cui cercare il dato, e il Tag viene confrontato in parallelo con i Tag memorizzati all'interno di tutte le linee del set selezionato: se uno dei confronti è positivo si ha un hit, altrimenti un miss.

Esiste una relazione inversa tra flessibilità e velocità nelle diverse organizzazioni: la **Direct Mapped** è la più veloce ma soffre di alti miss di conflitto; la **Fully Associative** elimina completamente i conflitti (ogni blocco può andare ovunque), ma ha l'Hit Time più alto e consuma più energia dovendo confrontare tutti i tag simultaneamente; la **Set Associative** bilancia questi due estremi. La complessità circuitale scala con l'associatività: mentre la mappatura diretta richiede un solo comparatore digitale, la Fully Associative richiede un comparatore per ogni linea, rendendola impraticabile per cache di grandi dimensioni. All'aumentare dell'associatività, il numero di set diminuisce e di conseguenza i bit dedicati all'Index diminuiscono, venendo assorbiti dal Tag: nel caso limite della Fully Associative, l'Index scompare del tutto e il processore utilizza l'intero indirizzo, escluso l'offset, come Tag per la ricerca associativa globale.

10.3 La Classificazione dei Miss e la Coerenza

I miss di cache si classificano secondo il modello delle **3C**. I miss **Compulsory** sono i miss fisiologici che avvengono al primo accesso assoluto a un blocco: non possiamo avere il dato se non l'abbiamo mai chiesto prima. I miss **Capacity** si verificano quando la cache non è sufficientemente grande per contenere tutti i blocchi necessari all'esecuzione corrente del programma, il cosiddetto *working set*. I miss **Conflict**, infine, accadono quando più blocchi competono per lo stesso set o riga anche se ci sarebbe spazio libero in altre parti della cache: sono tipici delle architetture Direct Mapped e si risolvono aumentando l'associatività.

Il tasso di miss varia con l'aumentare delle dimensioni del blocco: inizialmente riduce i miss obbligatori e di capacità sfruttando la località spaziale, fino a raggiungere una dimensione ottimale, oltre la quale le prestazioni degradano, perché blocchi troppo grandi riducono drasticamente il numero totale di linee nella cache, aumentando i miss di conflitto, e influenzano negativamente la latenza del recupero dati, aumentando la Miss Penalty.

La scrittura in cache pone un problema fondamentale di coerenza: abbiamo due copie dello stesso dato, una in cache e una in RAM, e quando la CPU modifica la copia in cache, quella in RAM diventa obsoleta. Le due strategie per gestire questo disallineamento sono opposte. Nel **Write-Through**, ogni volta che la CPU scrive in cache, il controller della memoria scrive immediatamente anche in RAM: cache e RAM sono sempre coerenti, ma il prezzo è la lentezza, dovendo aspettare i tempi della RAM per ogni singola scrittura. Nel **Write-Back**, invece, la CPU scrive solo nella cache, e l'aggiornamento del dato nella RAM avviene solo quando quel blocco di cache sta per essere cancellato per far posto a qualcos'altro (grazie all'uso di un *dirty bit*): il risultato è velocissimo, ma la complessità aumenta e c'è un rischio di incoerenza se salta la corrente o se un'altra periferica come il DMA legge la RAM vecchia.

10.4 Memoria Virtuale

La **memoria virtuale** simula una memoria di grandi dimensioni utilizzando lo spazio su disco, disaccoppiando gli indirizzi usati dal software (indirizzi virtuali) dagli indirizzi fisici della RAM. Questo meccanismo illude ogni processo di avere a disposizione una memoria contigua e molto estesa, indipendentemente dalla quantità di RAM fisica installata. Le caratteristiche principali sono il **paging**, che suddivide la memoria in blocchi a dimensione fissa chiamati pagine, e lo **swapping**, dove la RAM funge da cache per il disco: le pagine usate stanno in RAM, quelle non usate vengono spostate su disco nell'area di swap.

11 Multiply and Accumulate (MAC)

11.1 Il Prodotto Complesso e i suoi Componenti

I processori DSP sono progettati per eseguire algoritmi complessi come filtraggio, trasformate di Fourier e convoluzioni: questi algoritmi si basano su operazioni esprimibili come somme di prodotti, rendendo il MAC uno strumento indispensabile. Il prodotto complesso viene calcolato utilizzando la moltiplicazione binaria: questa tecnica divide l'operazione in prodotti parziali tra i bit di due numeri, sommati poi insieme propagando i riporti necessari. Per due numeri complessi x e y :

$$S = x + y = (x_r + y_r) + i(x_{im} + y_{im}), \quad P = x \times y = (x_r y_r - x_{im} y_{im}) + i(x_r y_{im} - x_{im} y_r)$$

Un **MAC a ciclo singolo** esegue somme e prodotti accumulati in un solo ciclo di clock: l'unità è composta da tre stadi in cascata (moltiplicatore, ALU, accumulatore) privi di registri intermedi, al fine di minimizzare la latenza e ottenere un throughput elevato. Grazie a questa architettura, un DSP può calcolare un "tap" di un filtro FIR per ogni ciclo di clock: se il processore gira a 100 MHz, può eseguire 100 milioni di MAC al secondo.

Il **MAC pipelined** suddivide la catena di elaborazione in sotto-catene sincrone, aggiungendo registri di pipeline tra le unità funzionali, in modo che il clock non debba più coprire i ritardi cumulativi ma solo il ritardo del singolo stadio più lento. Sebbene la latenza aumenti, il throughput cresce significativamente grazie alla frequenza più alta e al parallelismo, garantendo un flusso continuo di dati: mentre un primo stadio elabora un nuovo dato, i successivi processano i dati precedenti.

11.2 La Gestione della Crescita dei Bit

Nel design pipelined per numeri complessi, un aspetto cruciale è la gestione del cosiddetto **Bit Growth**, ovvero l'aumento della larghezza in bit dei dati man mano che attraversano il circuito, poiché occorre aumentare la precisione per non perdere dati. Si parte con dati a 16 bit (W_I , range $[-1, +1]$); quando si moltiplicano due numeri a 16 bit, il risultato ne richiede 32 (W_{PP} , Partial Products); sommando due numeri a 32 bit serve un bit in più per il riporto (W_{PC} , 33 bit, Complex Products); si ha poi uno stadio intermedio di troncamento (W_T , 20 bit); il risultato della somma di tutti i risultati parziali, con i cosiddetti *guard bits* che permettono al valore di crescere durante l'accumulazione, arriva a 22 bit (W_A , Accumulator); alla fine, il risultato torna a 16 bit (W_O , Output) tramite troncamento o arrotondamento.

La logica di overflow e saturazione viene gestita prima di scrivere il risultato finale in memoria: se il valore supera il range rappresentabile, il circuito applica la **saturazione**, forzando l'uscita al massimo valore positivo o negativo rappresentabile, invece di troncarsi i bit (che causerebbe un wrap-around e un'inversione di segno indesiderata).

11.3 La Moltiplicazione Binaria in Hardware

L'algoritmo classico di moltiplicazione “in colonna” applicato al sistema binario funziona così: ogni bit del moltiplicatore B genera un prodotto parziale che corrisponde a una copia del moltiplicando A se il bit è 1, o a una serie di zeri se il bit è 0. Ogni riga successiva viene fatta scorrere a sinistra per rispettare il peso posizionale, e infine si sommano tutte le colonne verticalmente. Il prodotto di due numeri a 4 bit richiede fino a 8 bit per essere rappresentato. Diverse versioni progressive di questa architettura (usando celle Half-Adder e Full-Adder in configurazioni diverse) mirano a uniformare progressivamente la struttura per ottenere massima regolarità e scalabilità, permettendo di collegare più moduli in cascata per gestire numeri a bitaggio superiore. Una tecnica avanzata, il **Folding**, sfrutta gli ingressi liberi nei blocchi superiori della matrice per reindirizzare i riporti, permettendo di eliminare l'intera riga finale di Full-Adder (*Vector Merging Adder*), riducendo drasticamente il numero di transistor necessari e accorciando il percorso critico.

12 Circuiti Digitali: Logica, Layout e Design Fisico

12.1 Dispositivi CMOS e Conduzione Complementare

I gate CMOS producono sempre o un 1 o uno 0, mai un valore indefinito. Per garantire ciò, le reti di **Pull-Up** (PUN) e **Pull-Down** (PDN) devono essere topologicamente duali: l'operazione AND si realizza con nMOS in serie più pMOS in parallelo, mentre per l'OR si usano nMOS in parallelo più pMOS in serie. Questa struttura garantisce che quando la rete di Pull-Down è accesa, quella di Pull-Up è spenta, e viceversa. La tecnologia CMOS permette di realizzare **Compound Gates**, cioè funzioni logiche complesse in un singolo stadio logico, risparmiando area e ritardo rispetto all'uso di porte base separate.

Il concetto di **signal strength** identifica quanto vicino un segnale si avvicina al valore ideale: gli nMOS trasmettono uno 0 forte e un 1 debole, rendendoli i migliori per i pull-down, mentre i pMOS trasmettono un 1 forte e uno 0 debole, rendendoli i migliori per i pull-up. Le **Transmission Gate**, combinando un blocco n e un blocco p in parallelo, fanno passare il segnale in modo bilanciato in entrambe le direzioni. Per evitare conflitti tra più dispositivi sulla stessa linea si usa un tri-state buffer, ma questa configurazione non presenta una logica di ristorazione del segnale: il rumore in ingresso si ripercuote sull'uscita. Per garantire l'integrità del segnale si utilizza il **Restoring Tristate Inverter**, in cui i due transistor centrali fungono da interruttori di abilitazione mentre i due esterni fungono da normale inverter CMOS,

rigenerando e ripulendo il livello logico dal rumore.

12.2 Logica Sequenziale a Livello Transistor

Il **D Latch** (trasparente) è modellabile come un multiplexer controllato dal clock che chiude un anello di retroazione: quando il clock è 1, il latch è trasparente e propaga il dato da D a Q; quando il clock è 0, il latch trattiene in Q l'ultimo valore visto, grazie a due inverter in cascata che si rigenerano continuamente. Il **D Edge-Triggered Flip Flop**, elemento base dei circuiti sincroni, è costruito da due D-Latch in serie controllati da fasi di clock opposte, in una configurazione **Master-Slave**: quando il clock è basso il Master è trasparente e campiona l'ingresso mentre lo Slave è bloccato; quando il clock è alto il Master si blocca e lo Slave diventa trasparente, trasferendo il dato all'uscita. Questo design “a camera stagna” impedisce race condition dovute al clock skew, in cui il dato potrebbe arrivare al secondo latch prima del clock, corrompendo la memoria.

12.3 Il Layout Fisico e le Design Rules

Disegnare un chip transistor per transistor sarebbe troppo lento per processori con miliardi di componenti: si usa quindi una libreria di mattoncini pre-fatti chiamati **Standard Cells**. Ogni cella ha dei contatti per polarizzare il substrato ed è contenuta in un rettangolo con altezza fissa e larghezza variabile in base alla complessità della funzione. Le linee di alimentazione VDD e GND devono combaciare tra celle adiacenti, formando un binario continuo; in alto si disegnano i pMOS (verso VDD), in basso gli nMOS (verso GND).

Il calcolo del “pitch” (passo) si basa sull'area occupata, definita dalla somma della larghezza del conduttore e dello spazio di isolamento necessario verso il conduttore adiacente. Esistono regole di design precise in unità λ per le spaziature interne (tra Metal e Diffusion, tra Metal e Polysilicon) e tra componenti (tra Metal, tra Diffusion, tra Polysilicon, tra Pull-Up e Pull-Down). Gli **Stick Diagram** rappresentano una visione topologica, non in scala, del circuito, che permette al progettista di pianificare il floorplan della cella ottimizzando il posizionamento dei componenti e il routing dei segnali prima del disegno dettagliato: le linee di metallo possono passare sopra poly e diffusione senza connettersi, e la connessione elettrica avviene solo in presenza di un contatto esplicito.

12.4 Il Physical Design del Processore

Il **Floorplanning** è il primo passo del physical design, dove si decide come disporre fisicamente i blocchi funzionali del sistema, per stimare l'area e la posizione dei blocchi principali. Il datapath, avendo struttura altamente regolare e ripetitiva, può essere progettato con la tecnica del **Bit-Slice**: si progetta il layout per un singolo bit e lo si replica per costruire l'intera unità. Il layout per il controller è invece irregolare e spesso implementato con standard cells; per collegare controller e datapath si usa il **Pitch Matching**, che allinea dimensioni e spaziatura delle celle adiacenti affinché le connessioni combacino.

Poiché la logica di controllo (FSM) è spesso irregolare, non è efficiente disegnarla con celle standard sparse: si utilizza una struttura regolare chiamata **PLA** (Programmable Logic Array), che mappa direttamente le equazioni booleane in un layout a griglia, implementando qualsiasi funzione combinatoria come “somma di prodotti” tramite due matrici adiacenti: un piano AND, che genera i mintermini, e un piano OR, che li combina per generare le uscite di controllo.

13 Affidabilità, Rumore e Variazioni di Processo

13.1 Process Corners

Mentre la litografia stampa il design delle maschere in modo diretto, il processo sul wafer ha un risultato di natura statistica, dipendente da tecnologia, ambiente e invecchiamento dei dispositivi. Non basta quindi progettare un circuito digitale che risponda ai valori nominali del processo: bisogna essere in grado di progettarlo entro dei parametri commerciali definiti **Process Corners**, condizioni limite entro le quali il progetto deve funzionare comunque, con l'idea ragionevole che se il circuito funziona bene ai corner, funzionerà bene anche al suo interno.

Si distinguono i corner **Front-End** (FEOL), che riguardano i contatti e la realizzazione fisica di base dei dispositivi (MOS), e i corner **Back-End** (BEOL), che riguardano la gestione dei dispositivi a livello superiore. La convenzione di denominazione per i corner FEOL è composta da due lettere: la prima si riferisce agli nMOSFET, la seconda ai pMOSFET, legate alla mobilità dei portatori di carica. Esistono cinque combinazioni: TT, FF, SS, FS, SF. TT, FF, SS sono chiamati *even corners* poiché influenzano uniformemente entrambi i tipi di MOS; FS e SF sono chiamati *skewed corners* poiché descrivono circuiti con prestazioni n-pMOS sbilanciate, una circostanza preoccupante a causa della commutazione asimmetrica che può causare un'errata memorizzazione dei dati nei latch. Per quanto riguarda il BEOL, si citano Process, Voltage e Temperature: la caratterizzazione definisce i circuiti come nominali (sezione media del processo), o come cbest/cworst (sezioni estreme ma ancora accettabili).

Esistono tecnologie di simulazione che, tramite metodi statistici, permettono di simulare le variazioni attese nel design su un certo processo, sfruttando spesso la tecnica di simulazione **Monte Carlo**, con variazioni di parametri random per analizzare se il circuito rimane dentro o meno i corner. Il processo non è uniforme: le distribuzioni di drogaggio non sono uniformi, e si possono avere variazioni spaziali tra lotti di wafer, tra wafer stessi, tra chip o all'interno del chip stesso, oltre a variazioni ambientali legate anche all'ambiente elettrico, come la sensibilità dell'alimentazione (tipicamente considerata al $\pm 10\%$).

13.2 Meccanismi di Guasto e Reliability

Le fonti di rumore hanno diverse sorgenti: dall'alimentazione, dalla massa, dalla carica scambiata capacitivamente tra linee adiacenti (crosstalk), fino al rumore termico intrinseco dei dispositivi. Un concetto centrale nell'affidabilità è quanto sia lungo lo **Useful Operating Life**, nella curva della reliability detta a "vasca da bagno" (*bathtub*): più è lunga, più è affidabile un prodotto, caratterizzato dal **Mean Time Between Failures** (MTBF) e dai **Failures in Time** (FIT). La probabilità di guasto è più alta nel periodo iniziale (mortalità infantile) e finale (usura) rispetto alla parte centrale della vita utile del componente. Per testare la vita dei prodotti senza dover attendere la loro intera durata reale, si ricorre all'**Accelerated Lifetime Testing**, sottoponendo i circuiti a situazioni estreme di temperatura e altre caratteristiche.

Gli **Hot Carriers** sono elettroni che acquisiscono dal campo elettrico un'energia cinetica molto superiore a quella di equilibrio termico: questo fenomeno si verifica quando il campo elettrico è sufficientemente intenso da accelerare gli elettroni lungo il loro libero cammino medio, permettendo loro di accumulare un'energia che non riescono a dissipare tramite gli urti con il reticolo cristallino. La problematica emerge quando questi elettroni ad alta energia interagiscono con l'ossido di gate, rimanendo intrappolati in stati spuri o difetti del dielettrico, causando una variazione progressiva della tensione di soglia del transistor.

Il **breakdown dell'ossido** (Time-Dependent Dielectric Breakdown, TDDb) è un processo di degradazione progressiva indotto dallo stress dei campi elettrici applicati: valori troppo elevati del campo elettrico nell'ossido (E_{OX}) accelerano la generazione di trappole e difetti

nel dielettrico, portando alla rottura definitiva anche a temperature standard; per garantire un'affidabilità a lungo termine è necessario limitare E_{OX} al di sotto di circa 0.7 V/nm. Il fenomeno **NBTI** (Negative Bias Temperature Instability) genera trappole in presenza di legami non strutturati, molto più diffuso nei p-MOSFET.

L'**elettromigrazione** è un fenomeno di usura dei conduttori causato dal passaggio di una forte corrente elettrica, dove il flusso degli elettroni colpisce gli atomi del metallo con una forza tale da spostarli fisicamente dalla loro posizione, un effetto noto come *vento elettronico*: questo spostamento di materia può svuotare alcune zone creando interruzioni (open circuit), oppure accumulare materiale altrove causando cortocircuiti. Il problema è particolarmente grave in regime di corrente continua, poiché gli elettroni spingono gli atomi sempre nella stessa direzione. La vita utile del conduttore, il **MTTF** (Mean Time To Failure), è descritta dall'**Equazione di Black**, che evidenzia come il guasto dipenda dalla densità di corrente e, in modo esponenziale, dalla temperatura di esercizio. Il fenomeno del **Self Heating** si verifica quando la corrente che attraversa le interconnessioni genera calore per effetto Joule, ostacolato nella dissipazione dallo strato di ossido di passivazione che agisce come coperta isolante, causando un aumento di resistenza e un conseguente rallentamento della propagazione dei segnali.

I guasti da sovratensione (**Overvoltage Failure**) si verificano quando una tensione eccessiva porta alla distruzione dei transistor, spesso a causa di scariche elettrostatiche ad alto voltaggio, richiedendo diodi di protezione sui pin e braccialetti di messa a terra per gli operatori. Il fenomeno del **Latch-up** riguarda una giunzione parassita che collega i blocchi nMOS e pMOS: se scorre corrente attraverso la resistenza del substrato, porta a un aumento di tensione e all'accensione di un transistor parassita, generando una retroazione positiva indesiderata che porta alla fusione dell'intero circuito; le soluzioni includono trench o guard ring diffusion attorno ai transistor. I **Soft Errors**, infine, sono malfunzionamenti casuali riscontrati frequentemente nelle memorie dinamiche, causati dall'interazione del silicio con particelle ad alta energia (particelle Alpha o neutroni dei raggi cosmici), che generano coppie elettrone-lacuna disturbando la tensione interna e portando all'inversione involontaria di un bit (*bit flip*). Per mitigare l'impatto dei soft error si può adottare la ridondanza, implementata ad esempio con celle a Dual Interlocked Feedback (DICE), oppure codici di correzione degli errori (ECC) a livello di sistema: questa strategia è nota come **Radiation Hardening**.

14 Test dei Circuiti Integrati (Design For Testability)

14.1 Il Rationale del Testing

I circuiti devono essere progettati, tra le altre cose, per essere testati facilmente, perché il costo dell'intervento di correzione aumenta drasticamente, di un ordine di grandezza pari a 10^4 , man mano che ci si allontana dalla fase di progetto verso la produzione e l'uso in campo. Per una validazione efficace è necessario avere una profonda conoscenza del sistema e sottoporlo a stress test in condizioni limite, con l'obiettivo di far emergere eventuali difetti latenti. Per gestire la complessità si applica l'approccio *divide and conquer*: il sistema viene scomposto in blocchi funzionali isolati, permettendo di testare separatamente ogni comportamento, garantendo la tracciabilità dei segnali (*observability*) per monitorare l'evoluzione interna del circuito e individuare esattamente dove si genera l'errore.

Il testing serve a diversi scopi: la **detection**, per determinare se il dispositivo è difettoso o meno; la **diagnosi**, per determinare cos'è difettoso, procedura molto costosa fatta solo se necessario per la correzione; la **caratterizzazione** su un campione della popolazione, per diagnosticare e correggere difetti di progettazione (con risultato grafico fondamentale lo *Shmoo Plot*, che visualizza le prestazioni dei dispositivi testati rispetto a frequenza e tensione); e l'analisi di **fault mode**, per determinare lacune nel processo di produzione.

Il **manufacturing test** determina quanto il circuito rispetta le specifiche, cercando il maggior numero di errori possibili senza porsi il problema di capire da dove venga il problema, ed è eseguito su tutti i circuiti prodotti. Lo **stress test** sottopone campioni di circuiti a condizioni estreme di temperatura e tensione per far emergere la cosiddetta mortalità infantile entro pochi giorni, mentre l'**incoming inspection** avviene dal lato del cliente, su lotti casuali, per bloccare componenti difettosi prima che vengano montati nei sistemi finali.

Legge/Teorema 14.1 (Production Yield).

$$Y = \frac{\text{numero chip buoni}}{\text{numero chip totali}}, \quad \text{Cost}_{\text{chip}} = \frac{\text{costo fabbricazione} + \text{costo test}}{Y \times N_{\text{chip/wafer}}}$$

14.2 Fault Modeling

I concetti chiave nel fault modeling sono la **controllabilità**, la capacità di controllare lo stato del sistema e porlo in una condizione specifica, e l'**osservabilità**, la capacità di propagare all'esterno il risultato di un nodo e di farlo vedere. Il modello di fault più diffuso è quello del **Single Stuck-at**: un certo nodo del circuito che dovrebbe essere in grado di muoversi liberamente rimane fisso a un valore, dovuto ad esempio a un difetto di fabbricazione. Tale difetto mantiene “ancorato” il valore del nodo a 0 (*stuck-at-0*, sa0) o a 1 (*stuck-at-1*, sa1); si definisce *single* perché si assume che al massimo sia presente una deformità di questo tipo nel circuito.

Il numero di *fault site* è uguale al numero di pin in ingresso sommato al numero di gate e al fan out. Attraverso il **fault collapsing**, si può ridurre il numero di vettori di test necessari eliminando fault site equivalenti che si possono dedurre da altri.

Legge/Teorema 14.2 (Checkpoint Theorem). *Si dimostra che è sufficiente rivelare gli stuck-at nei checkpoint (dati dai fan-in del circuito e dai rami dei gate con fan-out > 1) per capire il risultato della logica del circuito, riducendo drasticamente il numero di vettori di test necessari.*

Altri modelli di fault includono i **Bridging Faults**, dovuti al cortocircuito di nodi non adiacenti (errori di produzione); gli **Open Faults**, l'opposto dei bridging, dovuti alla mancanza di una connessione; e gli **IDD Faults**, dovuti a un cortocircuito verso VDD che porta a correnti in eccesso, un problema di tipo analogico piuttosto che logico.

14.3 Design For Testability

Si spende, a livello di area, per introdurre circuiti di testing dedicati, secondo tre categorie principali. L'**Ad Hoc Testing** usa circuiti non standard e difficili da implementare, aggiungendo dei test point tramite pad interni per facilitare la procedura di test su nodi altrimenti inaccessibili. Il **Scan Design** risolve la complessità degli Ad Hoc testing creando uno “scan-path” con il vantaggio di poter essere piazzato automaticamente dai software di sintesi, riducendo i tempi di test: si creano percorsi alternativi realizzati con registri a scorrimento per propagare gli stati interni del circuito verso un'uscita leggibile. La versione parallela dello scan design è il **Parallel Scan**, dove la lunga catena viene divisa in segmenti multipli caricati in parallelo; portando questo concetto al limite si ottiene il *random access scan*, che permette di indirizzare e accedere quasi singolarmente agli elementi di memoria, in modo simile al caricamento della memoria di configurazione nelle FPGA.

Il **BIST** (Built-In Self-Test) si basa tipicamente su **LFSR** (Linear Feedback Shift Register), registri generatori di sequenze pseudo-random seguendo il campo di Galois: con una retroazione positiva si ottengono sequenze randomiche con una periodicità anche molto lunga. Lo stato che si ottiene facendo passare tutti i dati attraverso questo meccanismo produce una firma digitale (hash) per tutti i dati che vi transitano; se prendiamo un circuito e lo passiamo attraverso il

generatore, otteniamo una firma digitale, e la probabilità che due circuiti randomici abbiano la stessa firma decresce con la radice quadrata del numero di registri ($1/\sqrt{2^{N_{reg}}}$). Un processore moderno ha un numero di stati interni così grande che è troppo complesso implementare questa logica in modo esaustivo, ma si può ottenere un payoff tipico del 99.7% di copertura con un consumo di circa il 10% di area aggiuntiva.

15 Memorie e Array

15.1 Static RAM

L'organizzazione della struttura **SRAM** è regolare, facile da disegnare e ad alta densità. La cella classica 6T è composta da due inverter incrociati (bistabile, che mantiene lo stato) più due transistor di accesso; il dimensionamento è cruciale: per la lettura serve $D_1/D_2 \gg A_1/A_2$ (i transistor di accesso), mentre per la scrittura serve $A_1/A_2 \gg P_1/P_2$, per evitare tensioni troppo alte sui nodi intermedi che potrebbero accendere/spegnere transistori indesiderati. Sono presenti due bitline che portano in uscita il dato in forma vera e negata, fondamentale per gli amplificatori di sense: essendo le capacità della bitline molto elevate, tramite l'uso degli amplificatori si può sbilanciare di appena un 10% della capacità massima per leggere uno zero o un uno, invece di caricare/scaricare completamente la linea.

15.2 Decoder e Memorie Grandi

Per un decoder con $N \leq 4$ basta una singola porta AND. Per decoder più grandi si usa una struttura gerarchica o di pre-decoding, che utilizza stadi multipli di porte NAND invece di una singola porta complessa, riducendo il numero di ingressi per singola porta (fan-in) e migliorando notevolmente la velocità di commutazione. Il **Predecoding** suddivide i bit di indirizzo in gruppi più piccoli per generare segnali intermedi prima dello stadio finale, riducendo l'occupazione di area. L'architettura **Hierarchical Wordlines** risolve i problemi di alta resistenza e capacità parassita delle lunghe interconnessioni suddividendo la decodifica in due livelli: Global Wordlines, che corrono su strati metallici superiori più larghi (bassa resistenza), e Local Wordlines, segmenti molto corti che pilotano direttamente le celle, migliorando le prestazioni riducendo il ritardo RC complessivo.

Le memorie grandi vengono partizionate in sotto-array per aumentare la velocità: più la memoria è piccola, più i decoder sono piccoli, più l'accesso risulta veloce. Le **RAM Multiport** offrono maggiore flessibilità tramite un maggior numero di porte che forniscono più valori alla cella, ad esempio con quattro porte di lettura e tre porte di scrittura simultanee.

15.3 Memorie ROM e DRAM

Nelle memorie **ROM**, l'interpretazione del valore logico dipende strettamente dalla configurazione circuitale dei transistor rispetto alla linea dati. Nell'architettura **NOR**, dove le celle sono disposte in parallelo, la linea viene mantenuta normalmente a un livello alto: la presenza di un transistor attivo crea un percorso di scarica verso terra, portando il valore a zero logico, mentre l'assenza di connessione lascia la linea alta (uno logico). Nella configurazione **NAND**, le celle sono collegate in serie, e la lettura richiede che tutti i transistor della catena, tranne quello selezionato, siano accesi per fungere da passaggi.

Le memorie **DRAM** (Dynamic RAM) immagazzinano l'informazione sotto forma di carica elettrica all'interno di un condensatore, accessibile tramite un singolo transistor di pass-gate controllato dalla wordline. Questa architettura essenziale (1T-1C) permette di ottenere densità di integrazione elevatissime, spesso realizzando il condensatore in profondità nel substrato (*trench capacitor*), ma costringe a un continuo refresh dei dati poiché la carica nel condensatore

tende naturalmente a disperdersi nel tempo. La lettura, inoltre, è distruttiva, e c'è la necessità di ricaricare la cella tramite gli amplificatori a valle.

15.4 Memorie Seriali

Uno **Shift Register** (registro a scorrimento) è un circuito sequenziale composto da una catena di flip-flop connessi in cascata e sincronizzati dallo stesso segnale di clock, dove l'uscita di ogni stadio è collegata all'ingresso del successivo, permettendo al dato binario di traslare di una posizione a ogni ciclo. Un tipico esempio è un registro a scorrimento a 4 bit con caricamento parallelo, che può funzionare in due modi:

```
always_ff @(posedge clk)
  if      (reset) q <= 0;
  else if (load)  q <= d;
  else       q <= {q[2:0], sin};
assign sout = q[3];
```

Se **reset**=1 il registro viene azzerato; se **load**=1 carica tutti i bit di **d** in un colpo solo; altrimenti esegue lo shift, spostando ogni bit verso il compagno più significativo e inserendo **sin** come nuovo bit meno significativo, mentre **sout** restituisce il bit più significativo in uscita dal registro. In VHDL la stessa logica si esprime con un segnale interno e l'operatore di concatenazione & al posto delle parentesi graffe.

Le **Queue** sono una struttura dati fondamentale che permette un accesso circolare alla memoria, basata su logica FIFO: dal punto di vista fisico viene realizzata con una SRAM che immagazzina i dati, due linee distinte per lettura e scrittura, e altre due per segnalare coda piena e vuota, gestita tramite puntatori alla memoria per tenere traccia di dove si è arrivati a scrivere e leggere.

16 Mappa Concettuale Riassuntiva

Concludiamo la dispensa con una mappa concettuale complessiva, che riassume visivamente le relazioni tra i macro-argomenti trattati, organizzati in sette macro-ambiti tematici principali:

- **Fondamenti Teorici e Tecnologici:** tecnologia CMOS/MOSFET, Macchina di Turing, complessità computazionale;
- **Sintesi HW/SW e Design Flow:** top-down flow, productivity gap, scheduling (AS-AP/ALAP), binding, curva di Pareto;
- **Linguaggi HDL e FPGA:** VHDL (entity/architecture, signal vs variable), FPGA (CLB, LUT, antifusibili, SRAM di configurazione);
- **Architetture di Processore:** MIPS single-cycle e multi-cycle, ISA, modalità di indirizzamento;
- **Memoria e Cache:** cache direct/set-associative, modello dei miss 3C, SRAM/DRAM/ROM, memoria virtuale;
- **DSP e Periferiche:** MAC, architettura Harvard, SPI/I2C, interrupt, DMA;
- **Affidabilità e Test:** process corners, hot carriers, TDDB, DFT, BIST.

Questi sette macro-ambiti, dai fondamenti tecnologici e teorici fino all'architettura dei microprocessori, alla gerarchia di memoria e alle tematiche trasversali di affidabilità e testabilità, costituiscono il percorso logico completo della disciplina dei Sistemi Embedded, dal livello più astratto della specifica comportamentale fino al layout fisico del transistor.